Sigurd: Algorithm Specifications

Version 1.0 (NGCC Submission)

Specification framework aligned with the NGCC submission structure.

Yi Deng[1], Yuanju Wei[2], Xinxuan Zhang[2], Zhuo Wu[2], Zhongliang Zhang[2], Lang Qin[2]

[1] School of Cryptology, Xidian University, Xi'an, China
[2] State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

ydeng.cas@gmail.com
{weiyuanju, zhangxinxuan, wuzhuo, zhangzhongliang, qinlang}@iie.ac.cn

September 21, 2026

Contents

List of Tables **4**

1 Introduction **5**

1.1 Technical background and motivation. 5

1.2 Security assumptions. 5

1.3 Two Main Technical Layers and Three Contributions 6

1.4 High-Level Overview of the Scheme 6

1.5 Differences from Prior VOLEitH-Based Designs 8

1.6 Document outline. 8

2 Preliminaries **8**

2.1 Notation 9

2.2 Data Types and Conversions 9

2.3 Cryptographic Primitives 10

2.4 Security Definitions 10

2.5 Reed–Solomon Codes 12

2.6 The Regular Syndrome Decoding Problem 13

3 Main Parameters **13**

3.1 Main Parameters 14

3.1.1 RSD Parameters 14

3.1.2 Extension-Field Parameters 14

3.1.3 Parameters for the Signature Scheme 14

3.1.4 Passing Parameters 14

3.1.5 Parameter Roles 15

3.1.6 Parameter Selection Rationale 15

4 GGM-Tree-Free VOLEitH Functions **17**

4.1 Vector Commitment with Linear-Relation Proofs 17

4.2 Compiler from Subfield VOLE to Vector Commitment 20

5 RSD Proof System Functions **22**

5.1 Lifted Representation of the RSD Instance 22

5.1.1 Tower Field Construction 23

5.1.2 Embedding $\mathbf{e}$ . . . 23
5.1.3 Embedding $\mathbf{H}$ . . . 23
5.1.4 Embedding $\mathbf{y}$ . . . 23
5.1.5 Syndrome Recovery Property . . . 23
5.2 IT-MACs Generation Phase . . . 24
5.3 Regularity Proof Protocol . . . 24
5.4 Σ-Protocol for $\mathbf{He} = \mathbf{y}$ . . . 26
5.5 Composing the Full Proof System . . . 27

6 The Sigurd Signature Scheme **28**
6.1 Algorithmic Overview . . . 28
6.2 Submitted Parameter Sets . . . 29
6.3 Encoding and Serialization . . . 29
6.3.1 Matrix, Witness, and Syndrome Derivation . . . 29
6.3.2 Fiat–Shamir Challenge Derivation . . . 30
6.3.3 Public and Secret Key Format . . . 31
6.3.4 Signature Format . . . 31
6.4 Internal Procedures . . . 32
6.5 Key Generation . . . 37
6.6 Signing . . . 37
6.7 Verification . . . 39

7 Performance Analysis **41**
7.1 Complexity Analysis . . . 41
7.2 Benchmark Results . . . 42

8 Security Evaluation **42**
8.1 Provable Security . . . 42
8.2 Concrete Security of the Underlying Hard Problem . . . 46

9 Scheme Characteristics and Potential Applications **46**
9.1 Algorithmic Features . . . 46
9.2 Potential Application Scenarios . . . 47

A Security Proofs for VC.LinearProof **49**

A.1 Knowledge Soundness 49
A.2 Special Honest-Verifier Zero Knowledge 51

B Proof of Compiler Security **52**

C Proofs for the RSD Proof System **54**
C.1 Embedding Preserves the Syndrome 54
C.2 Security of the Regularity Protocol 54
C.3 Security of the Σ-Protocol 55
C.4 Security of the Composed RSD Proof System 56

List of Tables

1 Roles of the main parameter families in Sigurd. 15
2 Submitted instance parameters for Sigurd. 29
3 Reference implementation benchmark results. 42
4 Optimized implementation benchmark results. 42
5 Classical security of the three RSD instances 46
6 Conservative quantum security of the three RSD instances 46

1 Introduction

This document specifies the Sigurd digital signature scheme, a post-quantum digital signature scheme whose security rests on the conjectured hardness of the *Regular Syndrome Decoding* (RSD) problem over $\mathbb{F}_2$.

Algorithm characteristics at a glance. Sigurd is built around a GGM-tree-free *VOLE-in-the-Head* (VOLEitH) proof system based on Reed–Solomon relation checks. This design removes the GGM tree [11] and SoftSpokenOT [14] layers of prior VOLEitH constructions, yielding a high-throughput RSD-based signature with larger signatures as the main tradeoff. In the optimized submitted implementation, signing takes about 1.8 ms, 10.7 ms, and 79.9 ms for NGCC-128, NGCC-256, and NGCC-512, respectively. Section 9.1 summarizes the corresponding algorithmic features.

1.1 Technical background and motivation.

The *VOLE-in-the-Head* (VOLEitH) paradigm, introduced by Baum et al. [3], converts a designated-verifier zero-knowledge proof built around a Vector Oblivious Linear Evaluation (VOLE) correlation into a publicly verifiable, non-interactive argument via the Fiat–Shamir transform [10]. Baum et al. also gave a post-quantum signature scheme (FAEST) based on the VOLEitH paradigm, using AES as the underlying one-way function. FAEST generates a VOLE correlation via a two-stage pipeline: first, a GGM-tree-based all-but-one commitment is used to commit the prover to the VOLE sender's inputs; then a SoftSpokenOT-style transformation converts the tree structure into the required VOLE outputs. This approach is ingenious while incurring many pseudorandom generator or hash function calls, because GGM trees and SoftSpokenOT transformations are all built on top of the pseudorandom generator or hash function.

Sigurd departs from this paradigm entirely. Rather than generating a VOLE correlation indirectly through an all-but-one commitment and an oblivious transfer transformation, Sigurd constructs the VOLE outputs *directly*: the prover commits to the relevant vectors $\mathbf{u}$ and $\mathbf{v}$ using a Reed–Solomon-based vector commitment scheme equipped with a proof of linear relation (derived from DEEP-FRI [4]), and then proves $\mathbf{w} = \Delta\mathbf{u} + \mathbf{v}$ in zero knowledge after receiving the verifier's challenge Δ. This direct approach eliminates both the GGM tree and the SoftSpokenOT layer, reducing the pseudorandom generator or hash function cost. Sigurd targets the RSD problem, which is a natural candidate for post-quantum security and has been the basis of several recent signature schemes [7, 8].

1.2 Security assumptions.

The EUF-CMA security of Sigurd in the random-oracle model is grounded in two independent assumptions:

- **RSD hardness.** Given a uniformly random parity-check matrix $\mathbf{H} \in \mathbb{F}_2^{(m-k)\times m}$ and syndrome $\mathbf{y} = \mathbf{He}$, it is computationally infeasible to recover the regular vector $\mathbf{e}$ satisfying the block-regularity condition $\mathrm{wt}(\mathbf{e}_i) = 1$ for all $i \in [n]$, where $\mathbf{e} = [\mathbf{e}_0 \| \cdots \| \mathbf{e}_{n-1}]$ and $n = m/B$.

The best known attacks against this problem are information-set decoding algorithms adapted to the regular block structure [9]; the submitted parameters are chosen to maintain a security margin against these.

- **Collision resistance.** The hash function H used for Merkle trees and commitment binding is modelled as a random oracle; however, its concrete instantiation must be collision-resistant in the standard model, namely, no efficient adversary can find distinct inputs $x \neq x'$ such that $\mathsf{H}(x) = \mathsf{H}(x')$ except with negligible probability.

No structured algebraic assumptions (lattices, elliptic curves, isogenies, etc.) are required at any level of the construction.

1.3 Two Main Technical Layers and Three Contributions

The construction of Sigurd is organized around the two technical layers that are developed in the sequel, namely the GGM-tree-free VOLEitH layer (Section 4) and the RSD proof-system layer (Section 5). Within these two layers, the scheme contributes three main technical ideas:

1. **A GGM-tree-free VOLEitH layer.** This layer replaces the standard GGM-tree and oblivious-transfer pipeline for realizing VOLE correlations. Its main technical contribution is:
   - The underlying sVOLE-admissible proof system is compiled into a non-interactive argument without any GGM tree or oblivious transfer. A Reed–Solomon vector commitment with DEEP-quotient-based linear-relation proofs directly generates the required VOLE correlation. The resulting prover complexity is $\Theta((n+\kappa)/\rho)$ hash calls, and knowledge soundness is inherited from the proximity gap theorem for Reed–Solomon codes [4].
2. **An RSD proof-system layer with two intertwined innovations.** Sigurd constructs a novel proof system for the RSD relation $\mathbf{He} = \mathbf{y}$ that is compatible with the GGM-tree-free VOLEitH layer, which has two intertwined technical contributions.
   - First, each one-hot block $\mathbf{e}_i$ of the regular witness is compressed into a single extension-field element $\hat{\mathbf{e}}_i = Y^{a_i} \in \mathbb{F}_{2^{\iota\tau}}$, so the regularity condition becomes the membership test $\hat{\mathbf{e}}_i \in \{1, Y, \ldots, Y^{B-1}\}$, enforced by an IT-MAC-based polynomial vanishing argument in the style of QuickSilver [17].
   - Second, after establishing authenticated shares of $\hat{\mathbf{e}}$, a dedicated masked-Σ protocol proves the syndrome relation by extracting the Y^{B-1} coefficient of $\hat{\mathbf{H}}\mathbf{w}_\mathbf{e}$ from the masked VOLE value $\mathbf{w}_\mathbf{e} = \Delta\hat{\mathbf{e}} + \mathbf{v}_\mathbf{e}$ without revealing the masking term.

1.4 High-Level Overview of the Scheme

Vector commitments with linear-relation proofs. The core building block of the Sigurd scheme is a vector commitment scheme equipped with a proof of linear relation, which is a variant of the DEEP-FRI [4]. In the concrete instantiation, the sender encodes the vectors $\mathbf{u} \in \mathbb{F}_p^\ell$ and $\mathbf{v} \in \mathbb{F}_{p^r}^\ell$ using Reed–Solomon codes, commits to the resulting codewords with Merkle trees, and additionally provides evaluations of the corresponding low-degree polynomials at a random point $z \notin \mathcal{L}$. These commitment values bind the sender to the underlying vectors while exposing only a compact digest to the receiver.

After the receiver issues a scalar challenge $\Delta \in \mathbb{F}_{p^r}$, the sender opens the induced linear combination $\mathbf{w}^* = \Delta \cdot \mathbf{u}^* + \mathbf{v}^*$ on the padded vectors. The receiver then checks a small set of queried codeword positions, verifies the associated Merkle openings, and tests that the claimed linear relation is consistent both locally and with a low-degree Reed–Solomon structure. This mechanism yields a compact proof that the committed vectors satisfy the desired linear relation, while providing knowledge soundness and special honest-verifier zero-knowledge (SHVZK) in the random-oracle model.

The GGM-tree-free VOLEitH framework. Recall that the goal of this VOLEitH layer is to ensure that the signer first commits to a pair of random vectors $\mathbf{u} \in \mathbb{F}_p^\ell$ and $\mathbf{v} \in \mathbb{F}_{p^r}^\ell$ before the challenge $\Delta \in \mathbb{F}_{p^r}$ is derived from the transcript. After learning Δ, the signer reveals the affine combination

$$\mathbf{w} = \Delta \cdot \mathbf{u} + \mathbf{v} \ \in \mathbb{F}_{p^r}^\ell$$

and provides a linear-relation proof showing that the revealed $\mathbf{w}$ is consistent with these values.

Standard FAEST-style constructions first build a GGM-tree-based all-but-one commitment, convert it via a SoftSpokenOT-style transformation to obtain the VOLE correlation, and then apply the QuickSilver proof system. This incurs $\Theta(nST)$ hash/PRG calls (where S is the challenge space size per repetition and T is the number of repetitions).

The Sigurd scheme instead employs the GGM-tree-free VOLEitH framework. The prover commits *directly* to the relevant vectors $\mathbf{u}$ and $\mathbf{v}$ using a vector commitment scheme with linear-relation proofs. After receiving the verifier's challenge Δ, the prover computes $\mathbf{w} = \Delta\mathbf{u} + \mathbf{v}$ and provides a proof that the opened $\mathbf{w}$ is consistent with the committed vectors. This avoids both the GGM tree and the SoftSpokenOT transformation, reducing the dominant prover cost to $\Theta((n+\kappa)/\rho)$ hash calls, where ρ is the RS code rate.

The lifted RSD proof system. The vector-commitment layer described above provides the VOLE correlation used by the proof system, but it does not by itself certify that the committed values encode a valid RSD witness. To prove knowledge of a witness for the relation

$$\mathcal{R}_{\mathrm{RSD}} = \{((\mathbf{H}, \mathbf{y}), \mathbf{e}) : \mathbf{He} = \mathbf{y},\ \mathbf{e}_i \in \mathbb{F}_2^B,\ \mathrm{wt}(\mathbf{e}_i) = 1 \text{ for all } i\}.$$

Sigurd lifts the binary instance to the tower field $\mathbb{F}_{2^{\iota\tau}}$ and proves validity of the lifted witness in three phases.

In this lifted representation, each one-hot block of the witness is compressed to a single extension-field element, and the matrix-vector relation is arranged so that the relevant syndrome information can be recovered from the designated Y^{B-1} coefficient. The proof system first uses the committed VOLE correlation to establish IT-MAC authenticated shares of the lifted witness and the auxiliary values needed for the later checks. It then proves regularity by showing that every lifted witness entry lies in the allowed set of monomials $\{1, Y, \ldots, Y^{B-1}\}$, which is equivalent to requiring that each binary block has Hamming weight exactly one.

Finally, a dedicated Σ-protocol verifies the lifted syndrome equation by checking that the Y^{B-1} coefficient of $\hat{\mathbf{H}}\hat{\mathbf{e}}$ equals the embedded syndrome $\bar{\mathbf{y}}$. Taken together, these components yield an sVOLE-admissible proof for the RSD relation, which is then compiled into the non-interactive signature protocol through the GGM-tree-free VOLEitH transformation of Section 4.

1.5 Differences from Prior VOLEitH-Based Designs

Sigurd follows the VOLE-in-the-Head methodology, but it is not a direct instantiation of prior FAEST-style VOLEitH signatures. FAEST [2] uses AES as the underlying one-way function and realizes the VOLEitH layer through GGM-tree commitments. ReSolveD [8] targets the RSD relation, but it also follows the standard GGM-tree and SoftSpokenOT pipeline for generating and opening the VOLE correlations. Sigurd keeps the RSD public-key relation as the object being proved, while modifying the way the VOLEitH layer is generated and checked.

The first core modification is the removal of the GGM-tree-based all-but-one commitment and the SoftSpokenOT-style transformation. Instead of deriving VOLE correlations from an all-but-one tree, the signer commits directly to the VOLE sender vectors and proves the required linear relation by using the Reed–Solomon vector-commitment mechanism described in Section 4. This change is local to the proof-generation layer: the public instance remains an RSD instance, and the signature still follows the Fiat–Shamir-with-aborts structure of VOLEitH signatures.

The second core modification is the lifted RSD proof system in Section 5. The binary block-regular witness is embedded into a tower field so that each one-hot block is represented by a single extension-field element. Regularity and syndrome consistency are then checked through field-level relations, rather than by proving the RSD witness bit by bit. This representation is tailored to the Reed–Solomon relation-checking layer and is the reason the VOLEitH modification can be combined with an RSD-based public-key relation.

These modifications are motivated by concrete efficiency. Removing the GGM tree and SoftSpokenOT-transformation reduces the amount of tree expansion and opening data handled by the signer and verifier, and the submitted implementation correspondingly prioritizes signing speed. The tradeoff is explicit: Sigurd obtains faster signing and verification with larger signatures. Thus the proposal should be viewed as a VOLEitH-based RSD signature design with a different correlation-generation and relation-checking, rather than as a parameter change of FAEST or ReSolveD.

1.6 Document outline.

Section 2 fixes notation, data types, the cryptographic primitives on which Sigurd relies, and the core mathematical objects used throughout the scheme. Section 3 introduces the main parameter framework. Section 4 specifies the vector commitment scheme with linear-relation proofs and the GGM-tree-free compiler. Section 5 first specifies the lifted representation of the RSD instance over the tower field and then presents the three-phase RSD proof system (IT-MACs generation, regularity proof, Σ-protocol). Section 6 gives the complete KeyGen, Sign, and Verify algorithms. Section 7 reports implementation benchmarks. Section 8 contains the formal security analysis. Section 9 closes with a discussion of the scheme's main characteristics and potential application scenarios.

2 Preliminaries

In this section we describe notation, basic data types and conversion algorithms, the core cryptographic primitives and security definitions used in Sigurd, and the mathematical objects on which the later protocols are built.

2.1 Notation

We use the following notation throughout.

Sets and sampling. We write $[a, b) = \{a, a+1, \ldots, b-1\}$ and $[a, b] = \{a, \ldots, b\}$, and $[n] = [1, n]$. We write $x \leftarrow S$ for uniform random sampling from a finite set S, and $x \xleftarrow{\$} \mathcal{A}$ for the output of a randomized algorithm $\mathcal{A}$. We write $\{x_i\}_{i \in S}$ for the set of all x_i with index in S.

Vectors and matrices. Bold lower-case letters $\mathbf{a}, \mathbf{b}, \ldots$ denote column vectors; bold upper-case letters $\mathbf{A}, \mathbf{B}, \ldots$ denote matrices. The i-th component of $\mathbf{a}$ (0-based) is $\mathbf{a}_i$ or a_i; the subvector with index set $[i, j]$ is $\mathbf{a}[i, j]$; the length of $\mathbf{a}$ is $\mathrm{len}(\mathbf{a})$. The Hamming weight of $\mathbf{a}$ is $\mathrm{wt}(\mathbf{a})$. Concatenation of two vectors or matrices of compatible dimensions is $[\mathbf{a} \| \mathbf{b}]$.

Fields and polynomials. $\mathbb{F}_q$ denotes the finite field with q elements. The polynomial ring over $\mathbb{F}_q$ is $\mathbb{F}_q[X]$; the quotient by a monic irreducible $f(X)$ of degree d is $\mathbb{F}_{q^d} \cong \mathbb{F}_q[X]/\langle f(X) \rangle$. Arithmetic in an extension field is always performed modulo the specified irreducible polynomial. $\mathbb{F}_q^{<d}[X]$ denotes the set of polynomials of degree strictly less than d.

Bit strings. A *bit string* of length t is an element of $\{0,1\}^t$. The bitwise XOR of two equal-length bit strings is $a \oplus b$. We write $\overline{b} := 1 - b$ for the complement of $b \in \mathbb{F}_2$.

2.2 Data Types and Conversions

Integer encoding. All integers are encoded in *little-endian* unsigned form. We write $\mathsf{LE16}$, $\mathsf{LE32}$, and $\mathsf{LE64}$ for 2-byte, 4-byte, and 8-byte encodings, respectively.

Bit packing. For a bit vector $(b_0, \ldots, b_{t-1}) \in \{0,1\}^t$, define

$$\mathsf{PackBits}(b_0, \ldots, b_{t-1})$$

to be the byte string whose j-th byte (0-based) is $\sum_{s=0}^{7} b_{8j+s} \cdot 2^s$, with missing high-index bits set to zero in the final byte.

Field-element encoding. Let the fixed polynomial basis of $\mathbb{F}_{2^\iota}$ be $1, X, \ldots, X^{\iota-1}$. An element $a = \sum_{t=0}^{\iota-1} a_t X^t \in \mathbb{F}_{2^\iota}$ is serialized as

$$\mathsf{Enc}_\iota(a) := \mathsf{PackBits}(a_0, \ldots, a_{\iota-1}).$$

With the fixed basis $1, Y, \ldots, Y^{\tau-1}$ over $\mathbb{F}_{2^\iota}$, an element $\alpha = \sum_{s=0}^{\tau-1} \alpha_s Y^s \in \mathbb{F}_{2^{\iota\tau}}$ is serialized as

$$\mathsf{Enc}_{\iota,\tau}(\alpha) := \mathsf{Enc}_\iota(\alpha_0) \| \mathsf{Enc}_\iota(\alpha_1) \| \cdots \| \mathsf{Enc}_\iota(\alpha_{\tau-1}).$$

Vectors over $\mathbb{F}_{2^\iota}$ or $\mathbb{F}_{2^{\iota\tau}}$ are serialized by concatenating component encodings in increasing index order.

2.3 Cryptographic Primitives

Hash functions and hash-based expansion. The scheme relies on hash-based primitives whose invocations are domain-separated. A fixed-output-length hash function H is used to instantiate Merkle-tree commitments; the binding property of the resulting commitments relies on the collision resistance of H. An instance-specific hash-based expansion procedure $\mathcal{H}$ is used for seed derivation, challenge generation, and deterministic bounded-range derivations such as witness-position generation. In concrete instantiations, these hash-based procedures may be implemented using a standard hash suite such as SHAKE128 or SHAKE256.

Definition 2.1 (Collision-Resistant Hash Function (CRHF))**.** A function $\mathsf{H} : \{0,1\}^* \to \{0,1\}^\lambda$ is *collision-resistant* if, for every polynomial-time adversary $\mathcal{A}$, the probability that $\mathcal{A}$ outputs (x, x') with $x \neq x'$ and $\mathsf{H}(x) = \mathsf{H}(x')$ is negligible in λ.

In this document $\mathcal{H}$ denotes the instance-specific hash-based expansion interface. All challenge derivation and seed expansion use $\mathcal{H}$ with domain-separation labels as defined in Section 6.3.2. The instance-specific hash suite (e.g. SHAKE128 or SHAKE256) is fixed in Table 2. In the security analysis, $\mathcal{H}$ is modelled as part of the programmable random-oracle interface used by the Fiat–Shamir transform and the associated seed-expansion procedures.

Definition 2.2 (Merkle Tree)**.** A *Merkle tree* over a leaf set $(\ell_0, \ldots, \ell_{N-1})$ is a complete binary tree in which each internal node is the hash of its two children, and the leaves hold $\mathsf{H}(\ell_i)$. The *root* is the single value at the top. A *Merkle authentication path* for leaf i consists of all sibling nodes on the path from leaf i to the root; it allows any party holding the root and the path to recompute and verify ℓ_i.

Information-theoretic MACs (IT-MACs). An IT-MAC over a finite field $\mathbb{F}_{p^r}$ is used internally to authenticate field elements. Let $\Delta \in \mathbb{F}_{p^r}$ be a global key (known only to the verifier). For a subfield or ambient field element $x \in \mathbb{F}$, where $\mathbb{F} \subseteq \mathbb{F}_{p^r}$, an IT-MAC on x is the pair $(K[x], M[x])$ satisfying

$$K[x] = M[x] + x \cdot \Delta,$$

where $K[x] \in \mathbb{F}_{p^r}$ is the verifier's global key share and $M[x] \in \mathbb{F}_{p^r}$ is the prover's tag. We write $[[x]] = (K[x], M[x], x)$ for the authenticated element. IT-MACs are *additively homomorphic*: given public coefficients $c_0, \ldots, c_{\ell-1} \in \mathbb{F}_{p^r}$, both parties locally compute $[[y]]$ where $y := \sum_{i=0}^{\ell-1} c_i \cdot x_i + c_\ell$, without interaction.

2.4 Security Definitions

Definition 2.3 (Digital Signature Scheme)**.** A *digital signature scheme* $\Pi = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ consists of three algorithms:

- $\mathsf{KeyGen}(1^\lambda) \to (\mathsf{pk}, \mathsf{sk})$. On input the security parameter 1^λ, the randomized algorithm KeyGen samples all secret internal randomness, generates the secret signing state, and derives the corresponding public verification information. It outputs a public key pk and a matching secret key sk.

- $\mathsf{Sign}(\mathsf{sk}, M) \to \sigma$. On input a secret key sk and a message $M \in \{0,1\}^*$, the possibly randomized algorithm Sign produces a signature σ. The signature is intended to certify that the holder of sk approved the message M.
- $\mathsf{Verify}(\mathsf{pk}, M, \sigma) \to \{\mathsf{accept}, \mathsf{reject}\}$. On input a public key pk, a message $M \in \{0,1\}^*$, and a candidate signature σ, the deterministic algorithm Verify checks whether σ is a valid signature on M under pk. It outputs accept if the check succeeds and reject otherwise.

The scheme is *correct* if

$$\mathsf{Verify}(\mathsf{pk}, M, \mathsf{Sign}(\mathsf{sk}, M)) = \mathsf{accept}$$

for all $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and all M.

Definition 2.4 (EUF-CMA Security)**.** A signature scheme $\Pi = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ is *existentially unforgeable under chosen-message attacks* (EUF-CMA) if, for every polynomial-time adversary $\mathcal{A}$, the probability

$$\Pr\Big[(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda);\ (M^*, \sigma^*) \leftarrow \mathcal{A}^{\mathsf{Sign}(\mathsf{sk},\cdot)}(\mathsf{pk}) : \mathsf{Verify}(\mathsf{pk}, M^*, \sigma^*) = \mathsf{accept} \ \wedge\ M^* \notin Q_{\mathsf{Sign}}\Big]$$

is negligible in λ, where Q_{Sign} denotes the set of messages queried by $\mathcal{A}$ to its signing oracle.

Definition 2.5 (Σ-Protocol)**.** A three-round public-coin protocol $(\mathcal{P}, \mathcal{V})$ for relation $\mathcal{R}$ is a *Σ-protocol* if its transcripts have the form (a, e, z), where a is the prover's first message, e is the verifier's uniformly sampled challenge, and z is the prover's response, and it satisfies:

- **Completeness**: for every $(x, w) \in \mathcal{R}$, on common input x and private input w to $\mathcal{P}$, an honest prover $\mathcal{P}$ always convinces an honest verifier $\mathcal{V}$.
- **Special soundness**: there exists a polynomial-time extractor Ext such that, for any statement x, any first message a, and any two accepting transcripts $(a, e, z), (a, e', z')$ for x with $e \neq e'$, the extractor outputs a witness
$$w \leftarrow \mathsf{Ext}(x, a, e, z, e', z')$$
such that $(x, w) \in \mathcal{R}$.
- **Special honest-verifier zero-knowledge (SHVZK)**: there exists a polynomial-time simulator that, given a statement x and an honestly sampled challenge e, it outputs an accepting transcript (a, e, z) that is indistinguishable from a real interaction.

Definition 2.6 (Subfield VOLE Functionality)**.** The ideal functionality $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$ is given in the following boxed form.

Algorithm 2.1. Functionality $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$

The functionality interacts with a sender $\mathcal{P}$, a receiver $\mathcal{V}$, and an adversary $\mathcal{A}$. It is parameterized by integers ℓ and p, r.

1. Upon receiving (init) from $\mathcal{P}$, sample $\mathbf{u} \xleftarrow{\$} \mathbb{F}_p^\ell, \mathbf{v} \xleftarrow{\$} \mathbb{F}_{p^r}^\ell, \Delta \xleftarrow{\$} \mathbb{F}_{p^r}$, and set $\mathbf{w} := \mathbf{v} + \Delta \cdot \mathbf{u} \in \mathbb{F}_{p^r}^\ell$.
   - If $\mathcal{P}$ is corrupted, receive $\mathbf{u}, \mathbf{v}$ from $\mathcal{A}$, and recompute $\mathbf{w}$ as above.
   - If $\mathcal{V}$ is corrupted, receive $\Delta, \mathbf{w}$ from $\mathcal{A}$, and compute and set $\mathbf{v} := \mathbf{w} - \Delta \cdot \mathbf{u}$.
   - Send $(\mathbf{u}, \mathbf{v})$ to $\mathcal{P}$ and send (done) to $\mathcal{V}$.

2. Upon receiving (get) from $\mathcal{V}$, send $(\Delta, \mathbf{w})$ to $\mathcal{V}$.

Definition 2.7 (sVOLE-Admissible Protocol)**.** An interactive proof protocol $\Pi = (\mathcal{P}, \mathcal{V})$ in the $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$-hybrid model is *sVOLE-admissible* if:

(1) $\mathcal{P}$ plays only the role of sender in $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$;

(2) the verifier $\mathcal{V}$ can be split into two phases $\mathcal{V}_1$ and $\mathcal{V}_2$, where $\mathcal{V}_1$ never sends get to $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$, and $\mathcal{V}_2$ only sends get.

One-way-to-hiding background. The one-way-to-hiding (O2H) theorem of Unruh [15] and the semi-classical O2H refinement of Ambainis, Hamburg, and Unruh [1] are standard tools for bounding the effect of hiding or reprogramming random-oracle values against quantum-query adversaries. In particular, this line of work formalizes the idea that a quantum adversary can detect a hidden random-oracle point only through its query amplitude on that point.

Theorem 2.8 (Semi-classical O2H [1, Theorem 1])**.** *Let $S \subseteq X$ be random, and let $G, H : X \to Y$ be random functions such that $G(x) = H(x)$ for all $x \notin S$. Let z be a random bit string, and allow (S, G, H, z) to have arbitrary joint distribution. Let $\mathcal{A}$ be an oracle algorithm of query depth d. Define*

$$P_{\text{left}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^H(z)], \qquad P_{\text{right}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^G(z)],$$

and let P_{find} be the probability of the event Find *in the corresponding punctured-oracle experiment. Then*

$$|P_{\text{left}} - P_{\text{right}}| \leq 2\sqrt{(d+1)P_{\text{find}}}.$$

Together with the optimality of Grover search for unstructured search problems [12, 5], this theorem supports the conservative conversion rule used in the parameter discussion: whenever the relevant classical attack component is dominated by generic search over a space of size about 2^s, the corresponding quantum component is accounted for at the square-root scale, about $2^{s/2}$.

The proofs in this document are stated in the programmable random-oracle model. When the same random-oracle interfaces are accessed by a quantum adversary, the O2H principle is used only as a conservative guideline for interpreting the resulting security level: hidden random-oracle points and generic search components are charged at the square-root scale. Consequently, the submitted parameter sets are chosen so that the claimed quantum security level is at least one half of the corresponding classical random-oracle security level.

2.5 Reed–Solomon Codes

Definition 2.9 (Reed–Solomon Code)**.** Let $\mathbb{F}$ be a finite field, $\mathcal{L} = \{L_1, \ldots, L_N\} \subseteq \mathbb{F}$ an evaluation domain, and $d \in \mathbb{N}$. The *Reed–Solomon code* $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$ is the set of functions $f : \mathcal{L} \to \mathbb{F}$ such that $f = \hat{f}|_{\mathcal{L}}$ for some $\hat{f} \in \mathbb{F}^{<d}[X]$. Its rate is $\rho := d/|\mathcal{L}|$.

The *encoding* map $\mathsf{Enc} : \mathbb{F}^d \to \mathbb{F}^N$ sends the coefficient vector $\mathbf{m} \in \mathbb{F}^d$ to its evaluation vector $(\hat{m}(L_1), \ldots, \hat{m}(L_N))$ where $\hat{m}(X) = \sum_{j=0}^{d-1} m_j X^j$. The minimum relative Hamming distance of the code is $\delta = 1 - \rho$.

Definition 2.10 (DEEP Quotient)**.** Let $z \in \mathbb{F} \setminus \mathcal{L}$ and $b \in \mathbb{F}$. For a function $f : \mathcal{L} \to \mathbb{F}$, the *DEEP quotient* $\mathsf{QUOTIENT}(f, z, b) : \mathcal{L} \to \mathbb{F}$ is defined by

$$g(x) = \frac{f(x) - b}{x - z}.$$

The function g is a codeword of $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d-1]$ if and only if f is a codeword of $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$ and $f(z) = b$.

Lemma 2.11 (Correlated Agreement Principle [13])**.** *Let $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$ be a Reed–Solomon code of rate $\rho = d/|\mathcal{L}|$, and let $\varepsilon \in (0, 1 - \sqrt{\rho})$ and $\delta \in [0, 1 - \sqrt{\rho} - \varepsilon]$. Suppose $\mathbf{a}, \mathbf{b} \in \mathbb{F}^{|\mathcal{L}|}$ have the following property: for a uniformly random scalar $\alpha \in \mathbb{F}$, the word $\alpha\mathbf{a} + \mathbf{b}$ is within Hamming distance $\delta|\mathcal{L}|$ of some codeword of $\mathcal{C}$ with non-negligible probability. Then there exist codewords $\mathbf{a}^*, \mathbf{b}^* \in \mathcal{C}$ such that $\mathbf{a}$ and $\mathbf{b}$ are each within Hamming distance $\delta|\mathcal{L}|$ of $\mathbf{a}^*$ and $\mathbf{b}^*$, respectively. Moreover, the agreement can be taken on a common set of coordinates of size at least $(1 - \delta)|\mathcal{L}|$.*

Lemma 2.12 (Quotient Equivalence [4])**.** *Let $\mathcal{L} \subseteq \mathbb{F}_q$, let $z \in \mathbb{F}_q \setminus \mathcal{L}$, let $d \geq 1$, and let $f : \mathcal{L} \to \mathbb{F}_q$ and $b \in \mathbb{F}_q$. Define $g = \mathsf{QUOTIENT}(f, z, b)$. Then for every $\delta \in (0, 1)$, the following are equivalent:*

- *there exists a polynomial $Q(X) \in \mathbb{F}_q[X]$ of degree at most $d - 1$ whose evaluation vector on $\mathcal{L}$ is within Hamming distance $\delta|\mathcal{L}|$ of g;*
- *there exists a polynomial $R(X) \in \mathbb{F}_q[X]$ of degree at most d such that $R(z) = b$ and whose evaluation vector on $\mathcal{L}$ is within Hamming distance $\delta|\mathcal{L}|$ of f.*

Lemma 2.13 (DEEP Uniqueness [13])**.** *Let $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$ be a Reed–Solomon code of rate ρ, let $\varepsilon \in (0, 1 - \sqrt{\rho})$, and let $\delta \in [0, 1 - \sqrt{\rho} - \varepsilon]$. For a uniformly random $z \in \mathbb{F} \setminus \mathcal{L}$ and any word $\mathbf{w} \in \mathbb{F}^{|\mathcal{L}|}$ and value $b \in \mathbb{F}$, except with negligible probability over the choice of z there exists at most one polynomial $P(X) \in \mathbb{F}[X]$ of degree at most $d - 1$ such that $P(z) = b$ and the evaluation vector of P on $\mathcal{L}$ is within Hamming distance $\delta|\mathcal{L}|$ of $\mathbf{w}$.*

2.6 The Regular Syndrome Decoding Problem

Definition 2.14 (Regular Syndrome Decoding (RSD) Problem)**.** Let m, k, B be positive integers with $m > k$ and $n := m/B$. The *Regular Syndrome Decoding problem* with parameters (m, k, B) is: given $\mathbf{H} \in \mathbb{F}_2^{(m-k)\times m}$ sampled uniformly and $\mathbf{y} := \mathbf{He}$ where $\mathbf{e} = [\mathbf{e}_0 \| \cdots \| \mathbf{e}_{n-1}]$ is sampled uniformly from $\{\mathbf{e} \mid \mathbf{e}_i \in \mathbb{F}_2^B,\ \mathrm{wt}(\mathbf{e}_i) = 1 \text{ for all } i \in [0, n)\}$, find $\mathbf{e}$.

The associated RSD relation is

$$\mathcal{R}_{\mathrm{RSD}} = \{((\mathbf{H}, \mathbf{y}), \mathbf{e}) : \mathbf{He} = \mathbf{y},\ \mathbf{e}_i \in \mathbb{F}_2^B,\ \mathrm{wt}(\mathbf{e}_i) = 1 \text{ for all } i\}.$$

The best known attacks against RSD are information-set decoding algorithms adapted to exploit the regular block structure. The concrete parameters in Table 2 are chosen to maintain security against the state-of-the-art algorithms.

3 Main Parameters

This section introduces the main parameter families used throughout Sigurd, building on the definitions fixed in Section 2. Concrete submitted parameter sets are collected later in Section 6.2, once the full signature construction has been specified.

3.1 Main Parameters

We let λ denote the computational security parameter. In addition to λ, the scheme specification uses parameters for the RSD instance, the tower extension field, the repeated Σ-protocol, and the vector-commitment/compiler layer.

3.1.1 RSD Parameters

The signature scheme is instantiated over an RSD instance as defined in Section 2.6. The main problem parameters are (m, k, B, n), where $n = m/B$. These parameters determine the parity-check matrix $\mathbf{H} \in \mathbb{F}_2^{(m-k)\times m}$, the syndrome length $m - k$, and the block-regular witness structure used throughout the proof system.

3.1.2 Extension-Field Parameters

To enable the compressed witness representation and VOLE-based proofs of Section 5.1, the scheme lifts the binary RSD instance to the tower field

$$\mathbb{F}_2 \subset \mathbb{F}_{2^\iota} \subset \mathbb{F}_{2^{\iota\tau}}.$$

The extension degrees ι and τ satisfy

$$\tau > B, \qquad \iota \cdot \tau \geq \lambda.$$

3.1.3 Parameters for the Signature Scheme

The signature scheme has two layers of proof. Each layer contributes a soundness error that must be made negligible:

- **Regularity proof**: soundness error $\varepsilon_{\text{reg}} = (B+1)/2^{\iota\tau}$, negligible when $\iota\tau \gg \log B$.
- **Syndrome Σ-protocol**: soundness error $\varepsilon_\Sigma = 2^{-\iota \cdot t_\Sigma} + 2^{-\iota\tau}$, where t_Σ is the number of parallel repetitions.
- **Vector-commitment linear-relation proof**: knowledge error $(1 - \delta_{\text{VC}})^{Q_{\text{VC}}} + \text{negl}(\lambda)$, where $\delta_{\text{VC}} \in [0, 1 - \sqrt{\rho} - \varepsilon_{\text{RS}}]$ is the proximity parameter used in Lemma 4.2, for a fixed Reed–Solomon slack parameter $\varepsilon_{\text{RS}} \in (0, 1 - \sqrt{\rho})$.

3.1.4 Passing Parameters

For this specification document, to avoid passing the fixed parameters explicitly to all algorithms, we define the following shorthand:

$$\texttt{param} := \big(\lambda,\ m,\ k,\ B,\ n,\ \iota,\ \tau,\ t_\Sigma,\ Q_{\text{VC}},\ \rho,\ N_{\text{VC}}\big).$$

3.1.5 Parameter Roles

The following table summarizes the role of each parameter family before the scheme is instantiated with concrete submitted values.

Table 1: Roles of the main parameter families in Sigurd.

Parameter family	Symbols	Role in the construction
Security target	λ	Computational target governing the soundness and concrete-security level of the submitted instance.
RSD parameters	(m, k, B, n)	Determine the binary parity-check matrix, syndrome length, block structure, and witness dimension.
Extension-field parameters	(ι, τ, f_1, g)	Determine the tower field $\mathbb{F}_{2^\iota} \subset \mathbb{F}_{2^{\iota\tau}}$ used for witness compression and embedded syndrome checks.
Σ-protocol parameters	t_Σ	Determine the number of parallel repetitions and the associated masking resources for the syndrome proof.
VC/compiler parameters	$(\rho, Q_{\mathrm{VC}}, N_{\mathrm{VC}})$	Determine the Reed–Solomon rate, query complexity, and evaluation-domain size of the vector-commitment linear-relation proof.
Committed-length parameters	$(\ell_{\mathrm{sig}}, m_{\mathrm{VC}})$	Determine the unencoded witness length and the padded length used by the commitment layer.
Hash/expansion parameters	$(\mathsf{H}, \mathcal{H})$	Determine transcript hashing, Merkle binding, seed expansion, and Fiat–Shamir challenge derivation.
Serialization parameters	$(\lvert\mathtt{pk}\rvert, \lvert\mathtt{sk}\rvert, \lvert\sigma\rvert)$	Determine concrete transmission and storage overhead for the submitted instance.

Remark 3.1. The submitted implementations use the hash and hash-expansion helper interfaces provided by the organizer template, rather than a separate custom hashing layer. The concrete digest widths and expansion-profile choices are instance-specific and are listed together with the submitted parameter sets in Section 6.2.

3.1.6 Parameter Selection Rationale

The submitted parameter sets are chosen jointly from the underlying RSD relation, the lifted proof system of Section 5, and the vector-commitment compiler of Section 4. The design objective is not to optimize each parameter in isolation, but to select compatible tuples that simultaneously preserve the hardness of the underlying RSD instance, achieve the required proof-system soundness, and keep the concrete implementation structurally uniform across the three submitted security levels.

RSD-level parameters. The block size B is a parameter of the regular syndrome decoding problem itself. In principle, the construction permits other values of B, provided that the resulting RSD instances satisfy the required security level and remain compatible with the lifted proof system. For the submitted instances, we fix $B = 6$, following the parameterization used in ReSolveD and the current RSD evaluation methodology. This choice allows the submitted instances to be benchmarked against the existing attack estimators for regular syndrome decoding, including the attack families represented by [9, 7, 6, 16]. The dimensions (m, k) are then selected so that the induced RSD instances meet the claimed classical and quantum security targets under these estimators.

Extension-field parameters. The lifting layer requires an extension field $\mathbb{F}_{2^{\iota\tau}}$ satisfying two structural conditions: $\tau > B$, so that the blockwise one-hot witness can be embedded as monomials $1, Y, \ldots, Y^{B-1}$ without interference from the reduction polynomial, and $\iota\tau > \lambda$, so that the ambient challenge and masking space has sufficient size for the target security parameter λ. In the submitted instances, the base-field degree is fixed to $\iota = 16$ across all security levels. This keeps the base arithmetic, bit-packing, and embedding logic uniform in the implementation, while the outer extension degree τ is adjusted across instances. Concretely, the choices $\tau \in \{10, 16, 32\}$ give ambient extension degrees $160, 256, 512$, respectively, thereby satisfying the required security thresholds while preserving a common implementation structure.

Parameters of the repeated Σ-protocol. The parameter t_Σ is the number of parallel repetitions of the syndrome Σ-protocol in Section 5.4. A single repetition contributes a soundness term of size $2^{-\iota}$, so parallel repetition is used to reduce this error to the required level. The same parameter also determines how many independent masking values must be reserved in the signer transcript, because each repetition uses its own masked auxiliary term. Accordingly, t_Σ is chosen jointly as a soundness parameter and as a transcript-shape parameter.

Vector-commitment soundness parameters. The vector-commitment linear-relation proof depends primarily on the Reed–Solomon code rate ρ and the query count Q_{VC}. The rate ρ controls the distance and therefore the knowledge soundness of the DEEP-FRI-style linear-relation proof: decreasing the rate improves the underlying proximity gap, but increases encoding length and concrete Merkle and opening costs. The query count Q_{VC} determines how many opened positions are checked by the verifier and enters directly into the knowledge-soundness bound of Lemma 4.2. The submitted values of ρ and Q_{VC} are therefore selected together so that the compiler layer achieves the target soundness while keeping the opening cost and signature size within the intended implementation range.

Commitment-shape parameters. The parameters N_{VC}, ℓ_{sig}, and m_{VC} are induced by the structure of the committed witness and by the vector commitment. The base witness contribution is the embedded regular witness, augmented by the auxiliary coordinates required by the repeated Σ-protocol and by the quotient-response layer. The final committed length is then padded to a form compatible with the selected Reed–Solomon code rate and the Merkle-tree organization used in the implementation. In particular, the tail coordinates are allocated by role: one part is reserved for the masked interactive checks, one part is reserved for the quotient-response relation, and the remaining part provides extra random tail coordinates for the commitment layer. The exact lengths are instance-specific, but the allocation principle is the same across the submitted parameter sets.

Irreducible polynomials. The lifting construction requires a degree-ι irreducible polynomial $f_1(X)$ for the base field and a degree-τ irreducible polynomial $g(Y)$ for the outer extension. At the abstract level, any such polynomials satisfying the structural separation condition of Section 5.1 would be valid. In the submitted instances, however, the concrete polynomials are chosen to be implementation-friendly: they are sparse and admit low-cost reduction, so that each eliminated high-degree term folds into a small number of lower-degree targets and requires as few base-field multiplications as possible. This criterion is especially important for the submitted degree-10, degree-16, and degree-32 outer extensions.

4 GGM-Tree-Free VOLEitH Functions

This section specifies the core building blocks of the GGM-tree-free VOLEitH framework used by Sigurd. Section 4.1 gives the vector commitment scheme with linear-relation proofs. Section 4.2 gives the compiler from sVOLE-admissible protocols to the NIZK in the random-oracle model.

4.1 Vector Commitment with Linear-Relation Proofs

Definition 4.1 (Vector Commitment with Linear-Relation Proof)**.** A *vector commitment scheme with linear-relation proof* consists of the following procedures:

- $\mathsf{VC.Setup}(1^\lambda, n)$: on input the security parameter and the vector length n, outputs public parameters pp.
- $\mathsf{VC.Commit}(\mathsf{pp}, \mathbf{x})$: on input pp and $\mathbf{x} \in \mathbb{F}^n$, outputs a commitment c_x and prover state st_x.
- VC.LinearProof: an interactive protocol between a prover and a verifier. The prover holds $(\mathsf{pp}, (\mathbf{x}, \mathbf{y}) \in (\mathbb{F}^n)^2, \Delta \in \mathbb{F}, \mathsf{st}_x, \mathsf{st}_y)$; the verifier holds $(\mathsf{pp}, (c_x, c_y), \mathbf{z} \in \mathbb{F}^n, \Delta)$. The verifier outputs accept or reject.

The scheme satisfies the following properties:

- **Completeness.** For any $\mathbf{x}, \mathbf{y} \in \mathbb{F}^n$ and $\Delta \in \mathbb{F}$, letting $\mathbf{z} = \Delta\mathbf{x}+\mathbf{y}$, if $(c_x, \mathsf{st}_x) \leftarrow \mathsf{VC.Commit}(\mathsf{pp}, \mathbf{x})$ and $(c_y, \mathsf{st}_y) \leftarrow \mathsf{VC.Commit}(\mathsf{pp}, \mathbf{y})$, then an honest execution of VC.LinearProof on common input $(\mathsf{pp}, c_x, c_y, \mathbf{z}, \Delta)$ and witness $(\mathbf{x}, \mathbf{y}, \mathsf{st}_x, \mathsf{st}_y)$ is accepted with probability 1.
- **Knowledge soundness.** For every PPT malicious prover $\mathcal{P}^*$, there exists an expected PPT extractor E such that, whenever $\mathcal{P}^*$ convinces the verifier to accept $(\mathsf{pp}, c_x, c_y, \mathbf{z}, \Delta)$ with non-negligible probability, the extractor E, given the same public input and oracle access to $\mathcal{P}^*$, outputs vectors $\mathbf{x}^*, \mathbf{y}^* \in \mathbb{F}^n$ satisfying that c_x and c_y are valid commitments to $\mathbf{x}^*$ and $\mathbf{y}^*$, respectively, and $\mathbf{z} = \Delta\mathbf{x}^* + \mathbf{y}^*$.
- **Special honest-verifier zero-knowledge (SHVZK).** For every PPT semi-honest verifier $\widehat{\mathcal{V}}$ that follows the honest protocol messages and uses honestly sampled challenge $\Delta \in \mathbb{F}$, there exists a PPT simulator $\mathsf{Sim} = (\mathsf{Sim}_0, \mathsf{Sim}_1)$ such that, for every security parameter λ, every $\mathbf{x}, \mathbf{y} \in \mathbb{F}^n$, and every PPT distinguisher $\mathcal{D}$, the simulated view produced by Sim is computationally indistinguishable from the real interaction of VC.LinearProof.

The concrete instantiation used in Sigurd is derived from DEEP-FRI. Its full protocol specification is given in Algorithm 4.1. At a high level, the commitment object combines two ingredients. First, the prover Reed–Solomon encodes the padded vectors and commits to the resulting codewords with Merkle trees. This binds the prover to evaluations on the full domain $\mathcal{L}$ while allowing later openings at a small query set. Second, the prover sends the evaluations $U(z)$ and $V(z)$ at a verifier-chosen point $z \notin \mathcal{L}$. These out-of-domain evaluations serve as anchors for the DEEP-style quotient test used in the linear-relation proof.

The role of padding is twofold. It statistically hides the unpadded vectors inside longer random codewords, which is needed for zero knowledge, and it ensures that the quotient polynomial checked by the verifier has degree at most $m-2$. The verifier never reconstructs the committed vectors in full; instead, it checks consistency between the Merkle-opened coordinates, the out-of-domain evaluations, and the Reed–Solomon structure implied by the quotient relation. The construction proceeds as follows.

Algorithm 4.1. Vector Commitment over $\mathbb{F}_{p^r}$ with Linear-Relation Proof

Parameters. $m := \ell + Q_{\mathrm{VC}} + r, \quad N := m/\rho, \quad$ evaluation domain $\mathcal{L} = \{L_1, \ldots, L_N\} \subseteq \mathbb{F}_{p^r}$, collision-resistant hash function H.

VC.Commit$(1^\lambda;\ \mathbf{u} \in \mathbb{F}_p^\ell,\ \mathbf{v} \in \mathbb{F}_{p^r}^\ell)$**:**

1. Sample $\mathbf{u}' \xleftarrow{\$} \mathbb{F}_p^{Q_{\mathrm{VC}}+r}$ and $\mathbf{v}' \xleftarrow{\$} \mathbb{F}_{p^r}^{Q_{\mathrm{VC}}+r}$; form $\mathbf{u}^* := \mathbf{u}\|\mathbf{u}'$ and $\mathbf{v}^* := \mathbf{v}\|\mathbf{v}'$, so $\mathbf{u}^* \in \mathbb{F}_p^m$ and $\mathbf{v}^* \in \mathbb{F}_{p^r}^m$.
2. Encode: $\tilde{\mathbf{u}} := \mathsf{Enc}(\mathbf{u}^*) \in \mathbb{F}_p^N$ and $\tilde{\mathbf{v}} := \mathsf{Enc}(\mathbf{v}^*) \in \mathbb{F}_{p^r}^N$ (Reed–Solomon codewords). Build Merkle trees over $(\tilde{u}_i, \tilde{v}_i)_{i\in[N]}$ (interleaved leaves) and send roots root_u, root_v.
3. Receiver $\mathcal{R}$: sample $z \xleftarrow{\$} \mathbb{F}_{p^r} \setminus \mathcal{L}$ and send z.
4. Via the natural embedding $\mathbb{F}_p \hookrightarrow \mathbb{F}_{p^r}$, view $\mathbf{u}^*$ and $\mathbf{v}^*$ as coefficient vectors of polynomials $U(X), V(X) \in \mathbb{F}_{p^r}[X]$ of degree at most $m-1$. Compute and send $U(z)$ and $V(z)$.
5. **Output**: commitments $c_u := (\mathsf{root}_u, U(z))$ and $c_v := (\mathsf{root}_v, V(z))$. Prover state is the randomness used to sample $(\mathbf{u}^*, \mathbf{v}^*, \tilde{\mathbf{u}}, \tilde{\mathbf{v}})$.

VC.LinearProof$(\mathcal{S}, \mathcal{R};\ c_u, c_v, \Delta)$**:**

1. Receiver $\mathcal{R}$: send Δ to $\mathcal{S}$.
2. Sender $\mathcal{S}$: compute $\mathbf{w}^* := \mathbf{u}^* \cdot \Delta + \mathbf{v}^*$ and $\tilde{\mathbf{w}} := \mathsf{Enc}(\mathbf{w}^*)$; send $\mathbf{w}^*$ to $\mathcal{R}$.
3. Receiver $\mathcal{R}$: compute $\tilde{\mathbf{w}} := \mathsf{Enc}(\mathbf{w}^*)$; sample $\mathcal{Q} \subseteq [N]$ of size Q_{VC} uniformly at random; request openings $\{(\tilde{u}_i, \tilde{v}_i)\}_{i\in\mathcal{Q}}$ with Merkle paths.
4. Sender $\mathcal{S}$: send $\{(\tilde{u}_i, \tilde{v}_i)\}_{i\in\mathcal{Q}}$ together with their Merkle authentication paths.
5. Receiver $\mathcal{R}$: parse commitments as $c_u = (\mathsf{root}_u, U(z))$ and $c_v = (\mathsf{root}_v, V(z))$.

(a) Verify Merkle paths for each $i \in \mathcal{Q}$; ensure all $\tilde{u}_i \in \mathbb{F}_p$. If any verification fails, **reject**.

(b) For each $i \in \mathcal{Q}$, check

$$\frac{\tilde{w}_i - (U(z) \cdot \Delta + V(z))}{L_i - z} = \Delta \cdot \frac{\tilde{u}_i - U(z)}{L_i - z} + \frac{\tilde{v}_i - V(z)}{L_i - z}.$$

If any equality fails, **reject**.

(c) Let $q_i := \big(\tilde{w}_i - (U(z) \cdot \Delta + V(z))\big)/(L_i - z)$ for each $i \in [N]$. Verify that $\mathbf{q} = (q_1, \ldots, q_N)$ is a valid Reed–Solomon codeword corresponding to a polynomial of degree at most $m - 2$. If not, **reject**.

6. If all checks pass, **accept**.

The commitment phase binds the sender to two low-degree polynomials $U(X), V(X) \in \mathbb{F}_{p^r}[X]$ whose coefficient vectors extend the target inputs $\mathbf{u}$ and $\mathbf{v}$. The linear-relation phase then certifies that the opened vector $\mathbf{w}^*$ is the coefficient vector of the polynomial $W(X) = \Delta U(X) + V(X)$. The verifier does not check this identity by reconstructing the full codewords. Instead, it verifies the identity on a random query set and simultaneously checks that the quotient

$$Q(X) := \frac{W(X) - (\Delta U(z) + V(z))}{X - z}$$

is itself a low-degree polynomial. This is exactly the condition that the claimed opening be compatible with a globally well-defined relation $W(X) = \Delta U(X) + V(X)$.

Operationally, the verifier's checks have three distinct purposes. The Merkle authentication paths guarantee that the queried symbols come from the committed codewords; the pointwise linear test guarantees local consistency with the claimed relation; and the Reed–Solomon proximity test on the quotient vector upgrades these local checks to a global statement about low-degree structure. This separation of roles is useful later in the signature scheme, where the same commitment object is reused to certify VOLE correlations non-interactively.

From a security perspective, the soundness intuition is the same as in DEEP-FRI-based proximity arguments. If a malicious sender could answer the verifier's queries successfully while opening an inconsistent value $\mathbf{w}^*$, then either the sender would have to violate Merkle binding on one of the queried positions, or it would have to make the quotient test pass for words that are far from any low-degree encoding. The former reduces to collision resistance of H, while the latter is ruled out by the sampling guarantee of the Reed–Solomon query test. Consequently, an accepting transcript can be turned into an extractor for vectors close to unique low-degree codewords, which in turn determines committed vectors $\mathbf{u}^*$ and $\mathbf{v}^*$ satisfying $\mathbf{w}^* = \Delta\mathbf{u}^* + \mathbf{v}^*$.

The zero-knowledge intuition comes from the randomized padding of $\mathbf{u}$ and $\mathbf{v}$. Because the committed codewords are encodings of padded vectors rather than the raw witness vectors themselves, the queried positions statistically hide the unpadded inputs, while the out-of-domain evaluations $U(z)$ and $V(z)$ can be simulated consistently for an honestly sampled challenge. The formal knowledge-soundness and SHVZK statements are given in Section 8; here we only record the structural reason that the protocol simultaneously supports extraction and simulation.

For knowledge extraction, we additionally rely on the standard straight-line extractability of Merkleized IOP commitments in the programmable random-oracle model, as in existing IOP-to-succinct-argument transformations; this is the commitment-layer extraction step used in Appendix A and later in the compiler proof [17].

Lemma 4.2 (Security of VC.LinearProof)**.** *For any $\varepsilon \in (0, 1-\sqrt{\rho})$ and $\delta \in [0, 1-\sqrt{\rho}-\varepsilon]$, and for any malicious sender $\mathcal{S}^*$, there exists an expected polynomial-time extractor E. Let* Acc *denote the event that $\mathcal{S}^*$ makes the verifier accept, and let* Ext *denote the event that $E^{\mathcal{S}^*}$ outputs a pair $(\tilde{\mathbf{u}}, \tilde{\mathbf{v}})$ satisfying the conditions below. Then*

$$\Pr[\mathsf{Ext}] \geq \Pr[\mathsf{Acc}] - (1-\delta)^{Q_{\mathrm{VC}}} - \mathrm{negl}(\lambda).$$

Whenever E outputs committed codewords $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{v}}$, they satisfy the following conditions except with negligible probability:

- *There exist unique vectors $\mathbf{u}^* \in \mathbb{F}_p^m$ and $\mathbf{v}^* \in \mathbb{F}_{p^r}^m$ such that $\mathrm{dist}(\mathsf{Enc}(\mathbf{u}^*), \tilde{\mathbf{u}}) < \delta N$ and $\mathrm{dist}(\mathsf{Enc}(\mathbf{v}^*), \tilde{\mathbf{v}}) < \delta N$. If $U^*(X), V^*(X)$ are the corresponding coefficient polynomials, then $U^*(z) = U(z)$ and $V^*(z) = V(z)$.*
- *For any vector $\mathbf{w}^*$ that the sender can send and still make the verifier accept with probability at least $(1-\delta)^{Q_{\mathrm{VC}}} + \mathrm{negl}(\lambda)$, it holds that $\mathbf{w}^* = \Delta\mathbf{u}^* + \mathbf{v}^*$.*

Proof. The detailed proof is given in Appendix A. At a high level, the extractor first recovers the committed codewords from the Merkle commitments. The verifier's quotient check then implies that these codewords are close to unique low-degree encodings consistent with the committed outside values $U(z), V(z)$. Therefore every accepting strategy with probability above the knowledge error yields extraction with probability at least acceptance probability minus $(1-\delta)^{Q_{\mathrm{VC}}} - \mathrm{negl}(\lambda)$. Finally, the same uniqueness argument forces any accepting opening to equal $\Delta\mathbf{u}^* + \mathbf{v}^*$. □

Lemma 4.3 (SHVZK of VC.LinearProof)**.** *In the programmable random-oracle model, the protocol VC.LinearProof satisfies special honest-verifier zero knowledge.*

Proof. The detailed proof is given in Appendix A.2. The simulator first programs the Merkle roots and outside values, then samples an opened vector and queried symbols with exactly the same affine distribution as in a real execution. The random-oracle programming is only used to complete the authentication paths consistently. □

4.2 Compiler from Subfield VOLE to Vector Commitment

This subsection specifies the compiler that turns an sVOLE-admissible protocol into a non-interactive proof system in the random-oracle model. The full compiler construction is given in Algorithm 4.2. Concretely, it replaces the subfield VOLE functionality $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$ in any sVOLE-admissible protocol Π with the vector commitment scheme of Section 4.1.

The compiler should be read as a generic transformation layer between an abstract proof protocol in the $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$-hybrid model and an implementable protocol that no longer assumes ideal VOLE access. Its main task is to replace each sender-side VOLE initialization by a pair of vector commitments to $(\mathbf{u}_{sid}, \mathbf{v}_{sid})$, and to replace each later receiver request for $(\Delta_{sid}, \mathbf{w}_{sid})$ by an explicit opening $\mathbf{w}_{sid} = \Delta_{sid}\mathbf{u}_{sid} + \mathbf{v}_{sid}$ together with a proof that this relation is consistent with the earlier commitments.

The requirement that Π be *sVOLE-admissible* is essential. It ensures that all verifier requests for VOLE outputs can be deferred to the end of the interaction, after the prover has already committed to its messages. Without this property, the verifier's challenge Δ_{sid} could influence earlier protocol

messages in a circular way, invalidating both soundness and the Fiat–Shamir compilation used by the final signature scheme.

Definition 2.7 captures this condition by requiring that the prover only acts as the sender in the ideal functionality and that the verifier can be decomposed as $\mathcal{V}_1$ followed by $\mathcal{V}_2$, where $\mathcal{V}_1$ performs the ordinary interaction and $\mathcal{V}_2$ only asks for the deferred VOLE outputs. Intuitively, $\mathcal{V}_1$ fixes the transcript prefix to which the prover is committed, and $\mathcal{V}_2$ merely instantiates the random linear checks needed to validate the committed VOLE relations. This is the precise analogue of the message ordering used in the original VOLEitH paradigm, but with vector commitments replacing the GGM-tree commitment layer.

Algorithm 4.2. Compiler from Subfield VOLE to Vector Commitment

Input. An sVOLE-admissible protocol $\Pi = (\mathcal{P}, \mathcal{V})$ in the $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$-hybrid model. Let $\mathsf{H} : \{0,1\}^* \to \{0,1\}^{2\lambda}$ be a collision-resistant hash function.

Construction of $\mathcal{P}_{\mathsf{Comp}}$: $\mathcal{P}_{\mathsf{Comp}}$ runs a copy of $\mathcal{P}$ and forwards all messages, with the following exceptions:

- When $\mathcal{P}$ sends init to $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$ with identifier sid, $\mathcal{P}_{\mathsf{Comp}}$ instead samples $\mathbf{u}_{sid} \xleftarrow{\$} \mathbb{F}_p^\ell$ and $\mathbf{v}_{sid} \xleftarrow{\$} \mathbb{F}_{p^r}^\ell$, and commits: $(\mathsf{com}_{sid}^u, \mathsf{com}_{sid}^v) \leftarrow \mathsf{VC.Commit}(\lambda, \mathbf{u}_{sid}, \mathbf{v}_{sid})$. $\mathcal{P}_{\mathsf{Comp}}$ sends $(sid, \mathsf{init}, \mathsf{com}_{sid}^u, \mathsf{com}_{sid}^v)$ to $\mathcal{V}_{\mathsf{Comp}}$ and provides $(\mathbf{u}_{sid}, \mathbf{v}_{sid})$ to $\mathcal{P}$.
- Upon receiving $(sid, \mathsf{get}, \Delta_{sid})$ from $\mathcal{V}_{\mathsf{Comp}}$, $\mathcal{P}_{\mathsf{Comp}}$ computes $\mathbf{w}_{sid} = \mathbf{v}_{sid} + \Delta_{sid} \cdot \mathbf{u}_{sid}$ and generates a proof π_{sid} using $\mathsf{VC.LinearProof}$. Then $\mathcal{P}_{\mathsf{Comp}}$ sends $(sid, \mathsf{get}, \mathbf{w}_{sid}, \pi_{sid})$ to $\mathcal{V}_{\mathsf{Comp}}$.

Construction of $\mathcal{V}_{\mathsf{Comp}}$: $\mathcal{V}_{\mathsf{Comp}}$ runs a copy of $\mathcal{V}$ and forwards all messages, with the following exceptions:

- Upon receiving $(sid, \mathsf{init}, \mathsf{com}_{sid}^u, \mathsf{com}_{sid}^v)$ from $\mathcal{P}_{\mathsf{Comp}}$, sends done to $\mathcal{V}$ acting as $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$.
- When $\mathcal{V}$ sends get to $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$, $\mathcal{V}_{\mathsf{Comp}}$ samples $\Delta_{sid} \xleftarrow{\$} \mathbb{F}_{p^r}$ and sends $(sid, \mathsf{get}, \Delta_{sid})$ to $\mathcal{P}_{\mathsf{Comp}}$.
- Upon receiving $(sid, \mathsf{get}, \mathbf{w}_{sid}, \pi_{sid})$ from $\mathcal{P}_{\mathsf{Comp}}$, verifies π_{sid} for $\mathbf{w}_{sid} = \mathbf{v}_{sid} + \Delta_{sid} \cdot \mathbf{u}_{sid}$ where $\mathbf{u}_{sid}$ and $\mathbf{v}_{sid}$ are committed in com_{sid}^u, com_{sid}^v. If verification passes, sends $(\Delta_{sid}, \mathbf{w}_{sid})$ to $\mathcal{V}$ acting as $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$; otherwise, aborts.

In the compiled protocol, the transcript naturally separates into two parts. The first part consists of the ordinary messages of the underlying hybrid-model protocol together with the commitment objects $(\mathsf{com}_{sid}^u, \mathsf{com}_{sid}^v)$. The second part consists of the deferred VOLE openings $(\Delta_{sid}, \mathbf{w}_{sid}, \pi_{sid})$. This separation is what allows the entire construction to be turned into a non-interactive argument in the random-oracle model: once the commitment phase is fixed, all verifier challenges can be derived from the transcript hash, and the prover responds with the corresponding linear-relation proofs.

In Sigurd, the compiler is applied to the RSD proof system of Section 5. The resulting non-interactive transcript is then serialized as part of the signature. Concretely, the commitment object $(\texttt{root_u}, \texttt{root_v}, U(z), V(z))$ and the proof object π_{lin} appearing later in Section 6 are exactly the

outputs produced by this compiler layer.

The security rationale of the compiler mirrors the two ingredients it combines. Completeness is preserved because an honest prover can locally sample $(\mathbf{u}_{sid}, \mathbf{v}_{sid})$, compute $\mathbf{w}_{sid} = \Delta_{sid}\mathbf{u}_{sid} + \mathbf{v}_{sid}$, and produce a valid linear-relation proof whenever the underlying hybrid-model protocol would have received a consistent ideal VOLE sample. Knowledge soundness is preserved because any successful cheating strategy must first produce accepting commitment openings, from which the extractor for the vector-commitment layer recovers the effective VOLE sender messages, and can then be passed to the extractor of the underlying sVOLE-admissible protocol. Likewise, SHVZK is preserved because the commitment layer has its own simulator for honest challenges, and the deferred VOLE outputs can therefore be simulated consistently with the simulator of the hybrid-model proof. The full reductions are deferred to Section 8.

Lemma 4.4 (Security of the Compiler)**.** *For any sVOLE-admissible interactive proof protocol* $\Pi = (\mathcal{P}, \mathcal{V})$ *in the* $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$*-hybrid model, the compiled protocol* $\mathsf{Comp}(\Pi) = (\mathcal{P}_{\mathsf{Comp}}, \mathcal{V}_{\mathsf{Comp}})$ *is an interactive proof protocol in the random-oracle model that satisfies:*

- Completeness*, if* Π *is complete;*
- Public-coin*, if* Π *is public-coin;*
- Knowledge soundness *with asymptotic error* $\kappa(\lambda) + \varepsilon_{\mathrm{VC}} + \mathrm{negl}(\lambda)$*, where* $\varepsilon_{\mathrm{VC}} = (1 - \delta_{\mathrm{VC}})^{Q_{\mathrm{VC}}}$*, if* Π *has a black-box knowledge extractor with knowledge error* $\kappa(\lambda)$*;*
- SHVZK*, if* Π *has a black-box SHVZK simulator.*

Proof. The detailed proof is given in Appendix B. The completeness and public-coin properties are immediate. Knowledge soundness is obtained by composing a commitment extractor for the vector-commitment layer with the extractor of the underlying sVOLE-admissible protocol, while SHVZK is obtained by combining the simulator of the hybrid-model protocol with the simulator for $\mathsf{VC.LinearProof}$. The additional $\varepsilon_{\mathrm{VC}}$ term comes from the extraction error of Lemma 4.2. □

5 RSD Proof System Functions

This section first specifies the lifted representation of the binary RSD instance used by the proof system and then gives the three-phase proof system for the RSD relation $\mathcal{R}_{\mathrm{RSD}}$. The three phases are: IT-MACs generation (Section 5.2), the regularity proof (Section 5.3), and the syndrome Σ-protocol (Section 5.4). Combining these yields the sVOLE-admissible protocol used as input to the compiler of Section 4.2.

5.1 Lifted Representation of the RSD Instance

Because a direct VOLE-based proof for $\mathbf{He} = \mathbf{y}$ over $\mathbb{F}_2$ would require authenticating $m = nB$ bit values, while only n authenticated values are needed (one per block), the scheme lifts the RSD instance to a tower extension field. This lifted representation is the common input format used by Algorithm 5.1, Algorithm 5.2, and Algorithm 5.3.

5.1.1 Tower Field Construction

Fix positive integers ι and τ with $\tau > B$ and $\iota \cdot \tau \geq \lambda$. Choose:

- a monic irreducible polynomial $f_1(X) \in \mathbb{F}_2[X]$ of degree ι;
- a monic irreducible polynomial $g(Y) \in \mathbb{F}_{2^\iota}[Y]$ of degree τ. Let d_2 be the second-highest degree appearing in $g(Y)$, we require
$$2B - 2 - \tau + d_2 < B - 1.$$

The tower extension field is

$$\mathbb{F}_2 \subset \mathbb{F}_{2^\iota} := \mathbb{F}_2[X]/\langle f_1(X)\rangle \subset \mathbb{F}_{2^{\iota\tau}} := \mathbb{F}_{2^\iota}[Y]/\langle g(Y)\rangle.$$

Each element of $\mathbb{F}_{2^{\iota\tau}}$ is identified with a coefficient matrix $\mathbf{T} \in \mathbb{F}_2^{\tau\times\iota}$ via $\sum_{s=0}^{\tau-1}\sum_{t=0}^{\iota-1} T_{s,t}X^tY^s$.

5.1.2 Embedding e

For a one-hot block $\mathbf{e}_i$ with the unique 1 at position $j \in \{0,\ldots,B-1\}$, set $T_{j,0} = 1$ and $T_{s,t} = 0$ for all other (s,t). The embedded block is thus the monomial $\hat{\mathbf{e}}_i = Y^j \in \mathbb{F}_{2^{\iota\tau}}$. The embedded witness vector is $\hat{\mathbf{e}} = (\hat{\mathbf{e}}_i)_{i\in[0,n)} \in (\mathbb{F}_{2^{\iota\tau}})^n$.

5.1.3 Embedding H

Partition $\mathbf{H}$ into $\mu := \lceil (m-k)/\iota \rceil$ row blocks and $C = n$ column blocks, yielding submatrices $\mathbf{H}^{(r,c)} \in \mathbb{F}_2^{\iota\times B}$ for $r \in [\mu]$, $c \in [n]$ (padding with zero rows as needed). Embed each $\mathbf{H}^{(r,c)}$ into $\mathbb{F}_{2^{\iota\tau}}$ via

$$T_{s,t} \leftarrow H^{(r,c)}_{\iota,\,B-1-s} \quad \text{for } s = 0,\ldots,B-1,\ t = 0,\ldots,\iota-1.$$

The result is $\hat{\mathbf{H}}^{(r,c)} = \sum_{s=0}^{B-1}\big(\sum_{t=0}^{\iota-1} H^{(r,c)}_{\iota,B-1-s}\cdot X^t\big)Y^s \in \mathbb{F}_{2^{\iota\tau}}$. Collect $\hat{\mathbf{H}} = (\hat{\mathbf{H}}^{(r,c)})_{r\in[\mu],c\in[n]}$.

5.1.4 Embedding y

Partition $\mathbf{y} \in \mathbb{F}_2^{m-k}$ into μ blocks $\mathbf{y}^{(r)} \in \mathbb{F}_2^{\iota}$ (padding with zeros). Each block is embedded into $\mathbb{F}_{2^\iota}$ via $U_t \leftarrow y_t^{(r)}$ for $t = 0,\ldots,\iota-1$. Write $\bar{\mathbf{y}} = (\bar{\mathbf{y}}^{(r)})_{r\in[\mu]}$ for the result.

5.1.5 Syndrome Recovery Property

Lemma 5.1 (Syndrome Recovery)**.** *Let $\mathbf{H} \in \mathbb{F}_2^{(m-k)\times m}$ and let $\mathbf{e} \in \mathbb{F}_2^m$ be a regular noise vector with $\mathbf{y} := \mathbf{He}$. Embed $\mathbf{H}$, $\mathbf{e}$, $\mathbf{y}$ as above. Then the $(B-1)$-th Y-coefficient of each entry of the product $\hat{\mathbf{H}}\cdot\hat{\mathbf{e}}$ equals the corresponding entry of $\bar{\mathbf{y}}$ in $\mathbb{F}_{2^\iota}$.*

Proof sketch. Each embedded block $\hat{\mathbf{e}}_i = Y^{a_i}$ for some $a_i \in \{0,\ldots,B-1\}$. The product $\hat{\mathbf{H}}^{(r,c)}\cdot\hat{\mathbf{e}}_c$ shifts the coefficients of $\hat{\mathbf{H}}^{(r,c)}$ by a_c positions in Y. The construction of $g(Y)$ ensures that the modular reduction by $g(Y)$ does not alter the Y^{B-1} coefficient, so the Y^{B-1} term of the sum $\sum_c \hat{\mathbf{H}}^{(r,c)}\hat{\mathbf{e}}_c$ directly gives the r-th block of $\bar{\mathbf{y}}$. See Appendix C.1 for the detailed argument. □

5.2 IT-MACs Generation Phase

The IT-MACs generation phase establishes authenticated shares of the embedded witness $\hat{\mathbf{e}}$ and the auxiliary variables under the global VOLE key Δ. Both parties have access to the subfield VOLE functionality $\mathcal{F}_{\mathsf{sVOLE}}^{2^{\iota\tau},1,n+B}$.

This phase is purely preparatory: it does not yet prove any semantic property of the witness, but it converts the prover's private values into authenticated shares that can later be manipulated homomorphically. The key observation is that once $\hat{\mathbf{e}}$ is authenticated under the global VOLE key Δ, all later checks can be phrased as polynomial identities over authenticated values. The correction $\mathbf{u_e} - \hat{\mathbf{e}}$ sent by the prover serves only to replace the random VOLE sender input with the actual embedded witness while preserving the affine relation needed for IT-MAC verification.

The full phase specification is given in Algorithm 5.1.

Algorithm 5.1. IT-MAC Generation

Public input: $\mathbf{H}, \mathbf{y}$. **Private input to $\mathcal{P}$:** $\mathbf{e}$ (and $\hat{\mathbf{e}}$ as computed by the embedding of Section 5.1).

1. $\mathcal{P}$ sends init to $\mathcal{F}_{\mathsf{sVOLE}}^{2^{\iota\tau},1,n+B}$. Upon receiving $(\mathbf{u}, \mathbf{v})$, $\mathcal{P}$ partitions:

$$\mathbf{u} = \mathbf{u_e} \parallel \mathbf{u}_{\text{lookup}} \parallel u_\Sigma,$$
$$\mathbf{v} = \mathbf{v_e} \parallel \mathbf{v}_{\text{lookup}} \parallel v_\Sigma,$$

   where $\mathbf{u_e} \in (\mathbb{F}_{2^{\iota\tau}})^n$, $\mathbf{u}_{\text{lookup}} \in (\mathbb{F}_{2^{\iota\tau}})^{B-1}$, $u_\Sigma \in \mathbb{F}_{2^{\iota\tau}}$. $\mathcal{P}$ computes and sends the correction $\mathbf{u_e} - \hat{\mathbf{e}}$ to $\mathcal{V}$.

2. $\mathcal{V}$, upon receiving done from $\mathcal{F}_{\mathsf{sVOLE}}^{2^{\iota\tau},1,n+B}$, sends get. After receiving $(\Delta, \mathbf{w})$ (where $\mathbf{w} = \Delta \cdot \mathbf{u} + \mathbf{v}$), $\mathcal{V}$ partitions $\mathbf{w}$ identically and computes

$$\mathbf{w}'_{\mathbf{e}} := \mathbf{w_e} - \Delta \cdot (\mathbf{u_e} - \hat{\mathbf{e}}).$$

   Thereafter $\mathbf{w}'_{\mathbf{e}} = \hat{\mathbf{e}} \cdot \Delta + \mathbf{v_e}$, providing an authenticated share of $\hat{\mathbf{e}}$.

Output. The parties jointly hold IT-MAC authenticated values $[[\hat{\mathbf{e}}_i]] = (\hat{\mathbf{e}}_i, v_{\mathbf{e},i}, w'_{\mathbf{e},i})$ for each $i \in [0, n)$, and analogous shares $[[\mathbf{u}_{\text{lookup}}]]$ and $[[u_\Sigma]]$.

5.3 Regularity Proof Protocol

The regularity proof checks that each embedded block satisfies $\hat{\mathbf{e}}_i \in \{1, Y, Y^2, \ldots, Y^{B-1}\}$, equivalently that $F(\hat{\mathbf{e}}_i) = 0$ for $F(x) := \prod_{j=0}^{B-1}(x - Y^j) \in \mathbb{F}_{2^{\iota\tau}}[x]$. Write $F(x) = a_0 + a_1 x + \cdots + a_B x^B$.

The full protocol specification is given in Algorithm 5.2.

Algorithm 5.2. Regularity Check

Precondition: The IT-MACs generation phase has been executed.

1. **Verifier's challenge.** $\mathcal{V}$ samples and sends a random vector

$$\mathbf{r} = (r_0, \ldots, r_{n-1}) \xleftarrow{\$} (\mathbb{F}_{2^{\iota\tau}})^n.$$

2. **Prover's coefficient extraction.** For each $i \in [0, n)$, $\mathcal{P}$ computes the polynomial evaluation

$$g(\mathbf{w}'_{\mathbf{e},i}) = g(\hat{\mathbf{e}}_i \cdot \Delta + v_{\mathbf{e},i}) = c_B^{(i)}\Delta^B + c_{B-1}^{(i)}\Delta^{B-1} + \cdots + c_0^{(i)}$$

using the QuickSilver technique. Since $F(\hat{\mathbf{e}}_i) = 0$, the leading coefficient vanishes: $c_B^{(i)} = F(\hat{\mathbf{e}}_i) = 0$.

$\mathcal{P}$ also computes an auxiliary polynomial encoding the lookup values:

$$D(\Delta) := \sum_{j=0}^{B-2} \big(u_{\text{lookup},j} \cdot \Delta + v_{\text{lookup},j}\big)\Delta^{B-2-j} = d_{B-1}\Delta^{B-1} + \cdots + d_0.$$

For each $j = 0, \ldots, B-1$, $\mathcal{P}$ computes

$$c_j := \sum_{i=0}^{n-1} r_i \cdot c_j^{(i)} + d_j$$

and sends $(c_0, \ldots, c_{B-1})$ to $\mathcal{V}$.

3. **Verifier's check.** $\mathcal{V}$ computes $\mathsf{fw}_i := F(w'_{\mathbf{e},i})$ for each i, then:

$$\mathsf{wc} := \sum_{i=0}^{n-1} r_i \cdot \mathsf{fw}_i + \sum_{j=0}^{B-2} w_{\text{lookup},j} \cdot \Delta^{B-2-j}.$$

$\mathcal{V}$ **accepts** if and only if

$$\mathsf{wc} = \sum_{j=0}^{B-1} c_j \cdot \Delta^j.$$

Otherwise $\mathcal{V}$ **rejects**.

In the $\mathcal{F}_{\mathsf{sVOLE}}^{2^{\iota\tau},1,n+B}$-hybrid model, the protocol in Algorithm 5.2 is complete, SHVZK, and has soundness error at most $(B+1)/2^{\iota\tau}$.

Lemma 5.2 (Security of the Regularity Protocol)**.** *In the $\mathcal{F}_{\mathsf{sVOLE}}^{2^{\iota\tau},1,n+B}$-hybrid model, the regularity protocol is complete and SHVZK, and has soundness error at most*

$$\varepsilon_{\text{reg}} = \frac{B+1}{2^{\iota\tau}}.$$

Proof. The detailed proof is given in Appendix C.2. Completeness follows by expansion of the polynomial identity $F(\hat{\mathbf{e}}_i) = 0$ for every block, SHVZK follows from the masking already present in the authenticated shares, and soundness follows from a Schwartz–Zippel style argument applied to the aggregated polynomial relation. □

5.4 Σ-Protocol for $\mathbf{He} = \mathbf{y}$

After the regularity proof, the syndrome relation $\mathbf{He} = \mathbf{y}$ is proved using the following Σ-protocol, with the row-block count μ defined above.

The purpose of this protocol is to recover exactly the Y^{B-1} coefficient of the masked product $\hat{\mathbf{H}}\mathbf{w}'_{\mathbf{e}}$ without revealing the remaining coefficients, which are polluted by the VOLE mask. The additional masked value r_Σ removes the last piece of information that would otherwise leak the unmasked linear combination of $\hat{\mathbf{H}}\hat{\mathbf{e}}$. The verifier therefore checks two facts simultaneously: first, that the extracted Y^{B-1} coefficient matches the embedded syndrome, and second, that the authenticated linear relation remains consistent with the global VOLE key.

Algorithm 5.3. Syndrome Σ-Check

Preconditions: IT-MACs generation and regularity proof phases completed.

1. **Prover's masking.** $\mathcal{P}$ samples $r_\Sigma \xleftarrow{\$} \mathbb{F}_{2^{\iota\tau}}$ with the Y^{B-1}-coefficient fixed to 0 and all other coefficients uniform in $\mathbb{F}_{2^\iota}$. $\mathcal{P}$ computes and sends the correction $u_\Sigma - r_\Sigma$ to $\mathcal{V}$.

2. **Verifier's challenge.** $\mathcal{V}$ samples and sends

$$\mathbf{r} = (r_0, \ldots, r_{\mu-1}) \xleftarrow{\$} (\mathbb{F}_{2^\iota})^\mu.$$

   (When multiplying $r_i \cdot h$ for $h \in \mathbb{F}_{2^{\iota\tau}}$, treat r_i as the constant polynomial $f_{r_i}(Y) = r_i$ and compute in $\mathbb{F}_{2^{\iota\tau}}$.)

3. **Prover's response.** $\mathcal{P}$ computes

$$\mathbf{h}_{\mathbf{e}} := \hat{\mathbf{H}}\hat{\mathbf{e}}, \qquad \mathbf{h}_{\mathbf{v}} := \hat{\mathbf{H}}\mathbf{v}_{\mathbf{e}},$$

   and forms:

$$h'_{\mathbf{e}} := \sum_{i=0}^{\mu-1} r_i \cdot h_{\mathbf{e},i} + r_\Sigma, \qquad h'_{\mathbf{v}} := \sum_{i=0}^{\mu-1} r_i \cdot h_{\mathbf{v},i} + v_\Sigma.$$

   $\mathcal{P}$ sends $(h'_{\mathbf{e}}, h'_{\mathbf{v}})$ to $\mathcal{V}$.

4. **Verifier's check.** Let $\bar{\mathbf{y}}$ be the embedding of $\mathbf{y}$ in $\mathbb{F}_{2^\iota}$ (see Section 5.1). $\mathcal{V}$ computes:

$$\bar{y}' := \sum_{i=0}^{\mu-1} r_i \cdot \bar{y}_i, \qquad \tilde{\mathbf{y}} := \hat{\mathbf{H}}\mathbf{w}'_{\mathbf{e}}, \qquad h_{\mathbf{w}} := \sum_{i=0}^{\mu-1} r_i \cdot \tilde{y}_i.$$

   $\mathcal{V}$ **accepts** if and only if *both*:

   (a) $\mathrm{coeff}_{Y^{B-1}}(h'_{\mathbf{e}}) = \bar{y}'$,

   (b) $h'_{\mathbf{e}} \cdot \Delta + h'_{\mathbf{v}} = h_{\mathbf{w}} + w_\Sigma - (u_\Sigma - r_\Sigma) \cdot \Delta$.

   Otherwise $\mathcal{V}$ **rejects**.

In the $\mathcal{F}_{\mathsf{sVOLE}}^{2^{\iota\tau},1,n+B}$-hybrid model, the protocol in Algorithm 5.3 is complete, SHVZK, and has

soundness error at most $2^{-\iota} + 2^{-\iota\tau}$. Running t_Σ parallel repetitions (each with fresh $\mathbf{w}_\Sigma$, $\mathbf{u}_\Sigma$, r_Σ) reduces the soundness error to $2^{-\iota \cdot t_\Sigma} + 2^{-\iota\tau}$.

Lemma 5.3 (Security of the Σ-Protocol)**.** *In the $\mathcal{F}_{\mathsf{sVOLE}}^{2^{\iota\tau},1,n+B}$-hybrid model, the Σ-protocol is complete and SHVZK, and has soundness error at most*

$$\varepsilon_\Sigma = 2^{-\iota} + 2^{-\iota\tau}.$$

Running t_Σ parallel repetitions reduces this to $2^{-\iota \cdot t_\Sigma} + 2^{-\iota\tau}$.

Proof. The detailed proof is given in Appendix C.3. Completeness is immediate from the syndrome recovery lemma and the correctness of the IT-MAC relation. SHVZK follows from the random masking term r_Σ and the uniformity of the authenticated auxiliary values. Soundness comes from the fact that a cheating prover must simultaneously guess the hidden Y^{B-1} coefficient and satisfy an independent field-valued linear relation. □

5.5 Composing the Full Proof System

Combined (SHVZK)PoK for RSD. The three phases above (IT-MACs generation, regularity proof, Σ-protocol) compose into a complete (SHVZK)PoK system for $\mathcal{R}_{\mathrm{RSD}}$ that is sVOLE-admissible. Applying the GGM-tree-free VOLEitH compiler (Section 4.2) yields a (SHVZK)PoK system in the random-oracle model. The Fiat–Shamir transform then yields the non-interactive signature scheme.

Theorem 5.4 (Security of the RSD Proof System)**.** *The protocol obtained by composing the IT-MACs generation phase, the regularity proof, and the t_Σ-fold parallel repetition of the Σ-protocol is a complete, public-coin, sVOLE-admissible* (SHVZK) proof of knowledge *for the relation $\mathcal{R}_{\mathrm{RSD}}$. Its knowledge error is bounded by*

$$\varepsilon_{\mathrm{reg}} + \varepsilon_\Sigma = \frac{B+1}{2^{\iota\tau}} + 2^{-\iota \cdot t_\Sigma} + 2^{-\iota\tau}.$$

Applying the compiler of Section 4.2 yields a complete, public-coin, interactive (SHVZK) proof of knowledge *for $\mathcal{R}_{\mathrm{RSD}}$ in the random-oracle model. Its knowledge error is bounded by*

$$\varepsilon_{\mathrm{reg}} + \varepsilon_\Sigma + \varepsilon_{\mathrm{VC}} + \mathrm{negl}(\lambda),$$

where $\varepsilon_{\mathrm{VC}} = (1-\delta_{\mathrm{VC}})^{Q_{\mathrm{VC}}}$ is the vector-commitment extraction error from Lemma 4.2.

Proof. The detailed proof is given in Appendix C.4. The IT-MAC generation phase exposes the witness in authenticated form, the regularity protocol proves blockwise one-hot structure, and the Σ-protocol proves the embedded syndrome equation. Since the verifier's VOLE requests occur only after the ordinary interaction, the composed protocol is sVOLE-admissible and can be compiled by Lemma 4.4. The hybrid-model extractor first obtains the embedded witness from the IT-MAC layer, then applies Lemma 5.2 and Lemma 5.3, and finally uses Lemma 5.1 to recover a valid binary RSD witness; this gives knowledge error $\varepsilon_{\mathrm{reg}} + \varepsilon_\Sigma$. The compiler then contributes the additional vector-commitment extraction error $\varepsilon_{\mathrm{VC}}$ together with a negligible random-oracle loss, yielding total knowledge error $\varepsilon_{\mathrm{reg}} + \varepsilon_\Sigma + \varepsilon_{\mathrm{VC}} + \mathrm{negl}(\lambda)$ for the compiled proof system. □

Optimization: direct commitment. Instead of committing to the full $\mathbf{u} = \hat{\mathbf{e}} \| \mathbf{u}_{\mathrm{lookup}} \| u_\Sigma$ and separately sending the correction $\mathbf{u_e} - \hat{\mathbf{e}}$, the signer commits directly to $\hat{\mathbf{e}} \| \mathbf{u}_{\mathrm{lookup}} \| r_\Sigma$. This reduces communication without compromising knowledge soundness or zero-knowledge.

6 The Sigurd Signature Scheme

This section instantiates the compiler layer of Section 4 and the lifted RSD proof system of Section 5 as the concrete signature scheme. The signer reconstructs the public matrix and the regular witness from compact seeds, embeds them into the tower field, executes the commitment, interactive-check, quotient-response, masking, and opening procedures in Fiat–Shamir order, and serializes the resulting transcript. The verifier replays exactly the same derivation chain and checks the same relations.

Supported usage domain. For the submitted digital-signature functionality, messages are arbitrary bit strings of length at most 2^{63} bits. The scheme is stateless at the key level: the public key and secret key remain fixed across signatures, and no per-key counter or one-time signing state is consumed. Accordingly, one key pair supports signing and verification for at least 2^{64} distinct messages, provided that each invocation of Sign uses fresh prover-side randomness for the commitment and masking procedures.

Correspondence with Sections 4 and 5. The concrete transcript fields of this section are the serialized realization of the proof objects defined earlier. The pair $(e_{\mathrm{eval}}, v_{\mathrm{eval}})$ together with the Merkle roots `root` instantiate the commitment object of the vector-commitment layer from Section 4. The arrays $(\mathtt{p_poly}_s, \mathtt{rv_out}_s)_{s<t_\Sigma}$ instantiate the interactive-check responses that combine the regularity proof and the embedded syndrome checks of Section 5. The response tuple $\mathtt{G} = (G_0, \ldots, G_5)$ is the concrete quotient-response object used by the compiler-side low-degree check. Finally, the masked opening vector $\hat{\mathbf{w}}' = \Delta\hat{\mathbf{e}}' + \hat{\mathbf{v}}'$ together with the opened symbols and authentication hashes is the serialized deferred VOLE opening that binds the Section 4 compiler layer to the lifted RSD witness of Section 5.

6.1 Algorithmic Overview

At a high level, the signature algorithm proceeds in five layers.

1. **State reconstruction.** From the compact seeds stored in the secret key, the signer reconstructs the binary matrix $\mathbf{H}$, the regular witness $\mathbf{e}$, the syndrome $\mathbf{y}$, and their tower-field embeddings. The verifier later reconstructs the same public-side objects from the public key.

2. **Commitment layer.** The signer commits to the embedded witness prefix together with random masking data, Reed–Solomon encodes the committed vectors, and builds the Merkle trees over the encoded symbols. Only after these Merkle roots are fixed does the signer derive the Fiat–Shamir point z and evaluate the committed witness and masking polynomials at that point.

3. **Consistency-proof layer.** Using the embedded products $\hat{\mathbf{H}}\hat{\mathbf{e}}$ and $\hat{\mathbf{H}}\hat{\mathbf{v}}$, the signer produces the interactive-check responses that implement the lifted regularity and embedded consistency checks described in Section 5.

4. **Polynomial-response layer.** The signer derives a second Fiat–Shamir challenge pair (τ, ζ) and computes the quotient-response polynomials that certify the low-degree relation required by the compiler layer of Section 4.

5. **Masked opening layer.** Finally, the signer derives the masking scalar Δ, forms the masked opening vector $\hat{\mathbf{w}}' = \Delta\hat{\mathbf{e}}' + \hat{\mathbf{v}}'$, derives the queried Merkle opening positions, and serializes the complete transcript. The verifier replays the same hash chain and checks these three objects in order: the commitment opening, the interactive checks, and the quotient-response identity.

6.2 Submitted Parameter Sets

The following table gives the concrete parameter sets of the three submitted instances. These values instantiate the parameter families introduced in Section 3.1.

Table 2: Submitted instance parameters for Sigurd.

Parameter	NGCC-128	NGCC-256	NGCC-512
Claimed classical security (bits)	128	256	512
Claimed quantum security (bits)	80	128	256
RSD dimensions (m, k)	$(1302, 738)$	$(2748, 1564)$	$(5676, 3230)$
Block size B, block count $n = m/B$	$B = 6,\ n = 217$	$B = 6,\ n = 458$	$B = 6,\ n = 946$
Tower-field degrees (ι, τ)	$(16, 10)$	$(16, 16)$	$(16, 32)$
Irred. poly. $f_1(X)$ for $\mathbb{F}_{2^\iota}$	$X^{16} + X^5 + X^3 + X^2 + 1$	$X^{16} + X^5 + X^3 + X^2 + 1$	$X^{16} + X^5 + X^3 + X^2 + 1$
Irred. poly. $g(Y)$ for $\mathbb{F}_{2^{\iota\tau}}$	$Y^{10} + Y^4 + Y^3 + XY + 1$	$Y^{16} + Y^{11} + Y + X$	$Y^{32} + Y^{31} + Y^5 + X$
Σ-protocol repetitions t_Σ	10	16	32
VC query count Q_{VC}	160	171	256
Reed–Solomon code rate ρ	1/4	1/8	1/16
Evaluation-domain size N_{VC}	1568	5208	19824
Augmentation tail length `TAIL_LEN`	175	193	293
Committed vector length $N' = n +$ `TAIL_LEN`	392	651	1239
Hash/expansion suite	SM3-256/ pseudoXOF	pseudohash-512/ pseudoXOF	pseudohash-1024/ pseudoXOF
Public-seed length (bits)	320	512	1024
Secret-seed length (bits)	320	512	1024
Encoded public-key length (bytes)	112	212	435
Encoded secret-key length (bytes)	80	128	256
Sample encoded signature length (bytes)	≈ 25108	≈ 74756	≈ 282692

6.3 Encoding and Serialization

6.3.1 Matrix, Witness, and Syndrome Derivation

Matrix derivation. The public matrix $\mathbf{H}$ is represented by a public seed `seed_H`. The implementation function `random_H` expands `seed_H` under an instance-specific label of the form `SIG128:H`, `SIG256:H`, or `SIG512:H`, and fills the binary matrix row by row with the resulting pseudorandom bits.

Witness derivation. The regular witness $\mathbf{e} \in \mathbb{F}_2^m$ consists of $n = m/B$ consecutive blocks of length B. In the submitted parameter sets, $B = 6$, so each block has Hamming weight 1. The implementation function `random_e` expands `seed_e` under an instance-specific label of the form `SIG128:e`, `SIG256:e`, or `SIG512:e`, reads one 16-bit word per block, and places the unique 1 of block i at position $\texttt{rnd}_i$ mod B. Equivalently, the witness may be described by block positions $a_0, \ldots, a_{n-1} \in \{0, \ldots, B-1\}$, so after embedding one has $\hat{\mathbf{e}}_i = Y^{a_i}$.

Syndrome derivation and packing. The syndrome is computed as $\mathbf{y} = \mathbf{He}$. In the reference implementation, the public key stores $\mathbf{y}$ packed in MSB-first order within each byte. Writing `packed_y` for the serialized syndrome,

$$\texttt{packed_y}[j] = \sum_{s=0}^{7} y_{8j+s} \cdot 2^{7-s}.$$

This is the convention implemented by `sig_pack_y_bits`.

Embedded state. The internal routine `sig_build_public_state` combines `random_H`, `random_e`, `generate_y`, `embed_H_raw`, `embed_e_raw`, `embed_y_raw`, and `Compute_YE_Raw`. It outputs:

- the binary matrix $\mathbf{H}$ and witness $\mathbf{e}$;
- the syndrome $\mathbf{y}$;
- the embedded matrix $\hat{\mathbf{H}}$;
- the embedded witness prefix $\hat{\mathbf{e}}'$;
- the packed syndrome coefficients $\bar{\mathbf{y}}$;
- the precomputed embedded product $\hat{\mathbf{y}}_{\mathbf{e}} := \hat{\mathbf{H}}\hat{\mathbf{e}}$.

The embedding routines follow the coordinate convention of Section 5.1. In `embed_e_raw`, the position a_i selected in block i is represented by setting the Y^{a_i} coefficient of the i-th RawGF2EX element to one, and the remaining committed tail positions are initialized to zero before the commitment phase samples the augmentation tail. In `embed_H_raw`, each $16 \times B$ binary row–column block of $\mathbf{H}$ is packed into one RawGF2EX element: for block position j, the corresponding 16 row bits are packed as the $\mathbb{F}_{2^{16}}$ coefficient of Y^{B-1-j}. In `embed_y_raw`, each syndrome row block is packed as one $\mathbb{F}_{2^{16}}$ scalar, so $\bar{\mathbf{y}}$ is the embedded syndrome used by the interactive checks and is distinct from the byte string `packed_y` stored in the public key. The implementation of `Compute_YE_Raw` multiplies $\hat{\mathbf{H}}$ only by the first $n = \texttt{COLS}$ embedded witness coordinates; the auxiliary tail coordinates are used later by the commitment and interactive-check procedures. Thus the signer-side and verifier-side high-level algorithms should be read as wrappers around this common state reconstruction logic.

6.3.2 Fiat–Shamir Challenge Derivation

The implementation derives all Fiat–Shamir challenges from a fixed-output hash Hash together with the domain-separated expander `derive_expand_bytes`. The logical challenge domains are:

Label suffix	Purpose
`H`	Matrix expansion from `seed_H`
`e`	Witness expansion from `seed_e`
`commitment`	Commitment evaluation point z
`intercheck`	Interactive-check challenges
`polyres`	Quotient-check challenges (τ, ζ)
`mask`	Masking scalar Δ
`open`	Merkle opening positions

For a message M, the transcript hash chain is:

$$\begin{aligned}
\mathsf{mh} &:= \mathsf{Hash}(M),\\
\alpha_{\mathrm{com}} &:= \mathsf{Hash}(\mathsf{mh} \| \mathtt{pk} \| \mathtt{root}),\\
\alpha_{\mathrm{ic}} &:= \mathsf{Hash}(\mathsf{mh} \| \alpha_{\mathrm{com}} \| e_{\mathrm{eval}} \| v_{\mathrm{eval}}),\\
\alpha_{\mathrm{poly}} &:= \mathsf{Hash}(\mathsf{mh} \| \alpha_{\mathrm{ic}} \| \mathtt{p_poly} \| \mathtt{rv_out}),\\
\alpha_{\mathrm{mask}} &:= \mathsf{Hash}(\mathsf{mh} \| \alpha_{\mathrm{poly}} \| \mathtt{G}),\\
\alpha_{\mathrm{open}} &:= \mathsf{Hash}(\mathsf{mh} \| \alpha_{\mathrm{poly}} \| \hat{\mathbf{w}}').
\end{aligned}$$

The helper functions `hash_to_rand_commitment`, `hash_to_rand_InterCheck`, `hash_to_rand_PolyRes`, `hash_to_rand_Mask`, and `hash_to_rand_Open` interpret the corresponding expansion streams as the required field elements, challenge arrays, and opening positions.

6.3.3 Public and Secret Key Format

$$\begin{aligned}
\mathtt{pk} &:= \mathtt{seed_H} \parallel \mathtt{packed_y},\\
\mathtt{sk} &:= \mathtt{seed_H} \parallel \mathtt{seed_e}.
\end{aligned}$$

In particular, the secret key stores both seeds explicitly; there is no master seed `seed_sk`.

6.3.4 Signature Format

The serialized signature produced by `sig_serialize_signature` is

$$\begin{aligned}
\sigma := {} & \mathtt{root}_0 \parallel \cdots \parallel \mathtt{root}_{g-1} \parallel \mathsf{Enc}(e_{\mathrm{eval}}) \parallel \mathsf{Enc}(v_{\mathrm{eval}})\\
& \parallel \mathsf{Enc}(\mathtt{p_poly}_0) \parallel \cdots \parallel \mathsf{Enc}(\mathtt{p_poly}_{t_\Sigma - 1})\\
& \parallel \mathsf{Enc}(\mathtt{rv_out}_0) \parallel \cdots \parallel \mathsf{Enc}(\mathtt{rv_out}_{t_\Sigma - 1})\\
& \parallel \mathsf{Enc}(G_0) \parallel \cdots \parallel \mathsf{Enc}(G_5)\\
& \parallel \mathsf{Enc}(\hat{\mathbf{w}}'_0) \parallel \cdots \parallel \mathsf{Enc}(\hat{\mathbf{w}}'_{N'-1})\\
& \parallel \mathsf{LE32}(h_{\mathrm{open}}) \parallel \mathtt{hash}_0 \parallel \cdots \parallel \mathtt{hash}_{h_{\mathrm{open}}-1}\\
& \parallel \mathsf{Enc}(e_0^{\mathrm{open}}) \parallel \cdots \parallel \mathsf{Enc}(e_{q-1}^{\mathrm{open}}).
\end{aligned}$$

The components correspond, respectively, to the Merkle roots, commitment evaluation values, interactive-check responses, quotient-response polynomials, masked opening vector, Merkle authentication hashes, and opened Reed–Solomon witness-code symbols. The opened-symbol block contains the queried encoded witness symbols only; the verifier reconstructs the corresponding masking symbols from $\hat{\mathbf{w}}'$, Δ, and the opened witness symbols before hashing the opened leaves.

6.4 Internal Procedures

This subsection records the internal procedures used by the concrete signing and verification algorithms. These procedures are not additional public APIs; they are the named internal interfaces invoked by the implementation and are defined here so that the full scheme description is explicit.

Algorithm 6.1. State-Derivation Procedures

`random_H`(`seed_H`)

- **Input:** public seed `seed_H`.
- **Output:** binary matrix $\mathbf{H}$.
- **Definition:**
  (a) derive a pseudorandom bit stream from `seed_H` under the instance label `*:H`;
  (b) read the stream in row-major order;
  (c) assign each bit to the next matrix entry until all rows and columns are filled.

`random_e`(`seed_e`)

- **Input:** secret seed `seed_e`.
- **Output:** regular witness $\mathbf{e} \in \mathbb{F}_2^M$.
- **Definition:**
  (a) derive a stream of 16-bit words from `seed_e` under the instance label `*:e`;
  (b) initialize the witness vector to zero;
  (c) for each block i, read the next word $\texttt{rnd}_i$ and set the unique nonzero position of block i to $\texttt{rnd}_i$ mod 6.

`generate_y`($\mathbf{H}, \mathbf{e}$)

- **Input:** binary matrix $\mathbf{H}$ and regular witness $\mathbf{e}$.
- **Output:** syndrome $\mathbf{y} = \mathbf{He}$ over $\mathbb{F}_2$.
- **Definition:** for each row of $\mathbf{H}$, compute the binary inner product with $\mathbf{e}$ and store the result as the corresponding syndrome bit.

`sig_build_public_state`(`seed_H`, `seed_e`)

- **Input:** public seed `seed_H`, secret seed `seed_e`.
- **Output:** binary state $(\mathbf{H}, \mathbf{e}, \mathbf{y})$ together with the embedded state $(\hat{\mathbf{H}}, \hat{\mathbf{e}}', \bar{\mathbf{y}}, \hat{\mathbf{y}}_{\mathbf{e}})$.
- **Definition:**

(a) invoke `random_H` to reconstruct $\mathbf{H}$;

(b) invoke `random_e` to reconstruct $\mathbf{e}$;

(c) invoke `generate_y` to compute $\mathbf{y}$;

(d) embed $\mathbf{H}$, $\mathbf{e}$, and $\mathbf{y}$ using `embed_H_raw`, `embed_e_raw`, and `embed_y_raw`;

(e) precompute $\hat{\mathbf{y}}_{\mathbf{e}} = \hat{\mathbf{H}}\hat{\mathbf{e}}$ using `Compute_YE_Raw`.

Algorithm 6.2. Challenge-Derivation Procedures

`derive_expand_bytes`(label, material, L)

- **Input:** domain-separation label label, input material material, output length L bits.
- **Output:** an L-bit expansion stream.
- **Definition:**

  (a) concatenate the label and the input material;

  (b) apply the instance hash-expansion interface to the concatenated string;

  (c) return the first L output bits.

`hash_to_rand_commitment`(α_{com})

- **Output:** commitment evaluation point z.
- **Definition:** interpret the expansion stream under label `*:commitment` as one field element.

`hash_to_rand_InterCheck`(α_{ic})

- **Output:** challenge vectors $(\mathbf{r}_s, \gamma_s)_{s<t_\Sigma}$.
- **Definition:** interpret the expansion stream under label `*:intercheck` as $((m-k)+1)\cdot t_\Sigma$ consecutive 16-bit words, using the first word of each block as γ_s and the remaining $m-k$ words as the entries of $\mathbf{r}_s$.

`hash_to_rand_PolyRes`(α_{poly})

- **Output:** quotient challenges (τ, ζ).
- **Definition:** interpret the expansion stream under label `*:polyres` as the vector τ and scalar ζ.

`hash_to_rand_Mask`(α_{mask})

- **Output:** masking scalar Δ.

`hash_to_rand_Open`(α_{open})

- **Output:** queried tree indices and leaf indices.
- **Definition:**
  (a) derive a sequence of 16-bit words from the label `*:open`;
  (b) interpret them as selections into the remaining unopened slots;
  (c) map the resulting slots to tree indices and leaf indices so that all opening positions are distinct.

Algorithm 6.3. Commitment Procedures

`Commitment_Raw`($\hat{\mathbf{e}}'$)

- **Input:** embedded witness prefix $\hat{\mathbf{e}}'$.
- **Output:** random masking vector $\hat{\mathbf{v}}'$, encoded vectors, Merkle tree data, and root tuple `root`.
- **Definition:**
  (a) sample a random augmentation tail of length `TAIL_LEN` and append it to the witness prefix;
  (b) force the first `SIGMA_REPETITIONS` tail elements to have zero Y^{B-1} coefficient;
  (c) sample a uniformly random masking vector $\hat{\mathbf{v}}'$ of full length N';
  (d) Reed–Solomon encode both vectors;
  (e) build the Merkle trees over the paired encoded symbols and return the root tuple.

`Commitment_Evals_Raw`($\hat{\mathbf{e}}', \hat{\mathbf{v}}', z$)

- **Input:** committed witness vector $\hat{\mathbf{e}}'$, masking vector $\hat{\mathbf{v}}'$, and commitment point z.
- **Output:** evaluations $(e_{\text{eval}}, v_{\text{eval}})$.
- **Definition:**
  (a) interpret $\hat{\mathbf{e}}'$ and $\hat{\mathbf{v}}'$ as coefficient vectors of the committed polynomials;
  (b) evaluate both polynomials at z by Horner-style accumulation;
  (c) return the resulting field elements e_{eval} and v_{eval}.

`Commitment_Open_Prover_Raw`($\mathtt{Hash}, \tilde{\mathbf{e}}, \tilde{\mathbf{v}}, \mathcal{Q}$)

- **Input:** Merkle tree hash storage, encoded witness and masking codewords, query set $\mathcal{Q}$.
- **Output:** opened encoded symbols and the corresponding authentication hashes.
- **Definition:**
  (a) determine the encoded coordinates requested by the opening queries;

(b) copy out the corresponding encoded witness and masking symbols;

(c) compute the minimal set of Merkle nodes required to authenticate all requested leaves in each tree;

(d) serialize these authentication hashes in tree order.

`Commitment_Open_Verifier_Raw`$(\hat{\mathbf{w}}', z, e_{\text{eval}}, v_{\text{eval}}, \Delta, \mathcal{Q})$

- **Input:** masked opening vector $\hat{\mathbf{w}}'$, evaluation point z, commitment evaluations, masking scalar Δ, and opening data for query set $\mathcal{Q}$.
- **Output:** acceptance bit and reconstructed encoded codeword $\tilde{\mathbf{w}}$.
- **Definition:**

(a) evaluate the polynomial represented by $\hat{\mathbf{w}}'$ at z;

(b) verify the evaluation identity $w_{\text{eval}} = \Delta e_{\text{eval}} + v_{\text{eval}}$;

(c) Reed–Solomon encode $\hat{\mathbf{w}}'$ to obtain the codeword used in the opening checks;

(d) verify that the opened encoded symbols match the queried coordinates of the encoded codewords;

(e) recompute the Merkle roots from the opened leaves and authentication hashes and compare them with the transmitted roots.

Algorithm 6.4. Proof Procedures

`Initial_Computation_Prover_Raw`$(\hat{\mathbf{H}}, \hat{\mathbf{v}}')$

- **Output:** $\hat{\mathbf{y}}_{\mathbf{v}} := \hat{\mathbf{H}}\hat{\mathbf{v}}'$.
- **Definition:** multiply each embedded row block of $\hat{\mathbf{H}}$ by the masking vector and reduce modulo the tower-field defining polynomial.

`Initial_Computation_Verify_Raw`$(\hat{\mathbf{H}}, \hat{\mathbf{w}}')$

- **Output:** $\hat{\mathbf{y}}_{\mathbf{w}} := \hat{\mathbf{H}}\hat{\mathbf{w}}'$.
- **Definition:** compute the embedded row products needed by the verifier-side interactive checks.

`Interactive_Check_Raw_Prover`$(\hat{\mathbf{y}}_{\mathbf{e}}, \hat{\mathbf{y}}_{\mathbf{v}}, \hat{\mathbf{e}}', \hat{\mathbf{v}}', \bar{\mathbf{y}}, (\mathbf{r}_s, \gamma_s))$

- **Output:** response pairs $(\texttt{p_poly}_s, \texttt{rv_out}_s)_{s<t_\Sigma}$.
- **Definition:**

(a) for each repetition s, aggregate the syndrome blocks of $\hat{\mathbf{y}}_{\mathbf{e}}$ and $\hat{\mathbf{y}}_{\mathbf{v}}$ with the random coefficients $\mathbf{r}_s$;

(b) aggregate the corresponding binary syndrome bits of $\bar{\mathbf{y}}$ and embed the result in the fixed Y^{B-1} coefficient slot;

(c) multiply the designated tail coordinates by γ_s and add them to the aggregated values;

(d) output the resulting pair $(\mathtt{p_poly}_s, \mathtt{rv_out}_s)$.

$\mathtt{Interactive_Check_Raw_verify}(\hat{\mathbf{y}}_{\mathbf{w}}, \hat{\mathbf{w}}', \bar{\mathbf{y}}, \Delta, (\mathbf{r}_s, \gamma_s), (\mathtt{p_poly}_s, \mathtt{rv_out}_s))$

- **Output:** acceptance bit.
- **Definition:**

(a) recompute the aggregated embedded linear forms from $\hat{\mathbf{y}}_{\mathbf{w}}$ and the random coefficients $\mathbf{r}_s$;

(b) recompute the embedded binary coefficient in the fixed Y^{B-1} slot;

(c) check the response identity linking $\mathtt{p_poly}_s$, $\mathtt{rv_out}_s$, Δ, and the designated masked tail coordinate;

(d) check that the Y^{B-1} coefficient of $\mathtt{p_poly}_s$ is zero.

$\mathtt{PolyDef_Raw}(\,)$

- **Output:** the fixed base polynomial coefficients used in the quotient relation.
- **Definition:** return the fixed coefficient tuple of the base polynomial used by the quotient-response identity.

$\mathtt{ProverPolynomialResponse_Final_Raw}(\hat{\mathbf{e}}', \hat{\mathbf{v}}', \tau, \zeta)$

- **Output:** quotient-response polynomials $G_0, \ldots, G_5$.
- **Definition:**

(a) evaluate the prover-side polynomial expression determined by the base polynomial, the challenge vector τ, the scalar ζ, the witness prefix, the masking vector, and the designated tail blocks;

(b) package the resulting coefficients into the six response polynomials $G_0, \ldots, G_5$.

$\mathtt{Masking_Setup_Raw}(\Delta, \hat{\mathbf{e}}', \hat{\mathbf{v}}')$

- **Output:** masked opening vector $\hat{\mathbf{w}}' := \Delta\hat{\mathbf{e}}' + \hat{\mathbf{v}}'$.
- **Definition:** multiply every coordinate of the witness prefix by Δ and add the corresponding masking coordinate.

$\mathtt{Polynomial_Challenge_Raw}(\tau, \zeta, \hat{\mathbf{w}}', \Delta)$

- **Output:** verifier-side polynomial value $\mathtt{vc}_{\mathrm{lhs}}$.

- **Definition:** evaluate the verifier-side polynomial expression determined by τ, ζ, $\hat{\mathbf{w}}'$, the designated tail portion, the base polynomial coefficients, and Δ.

`VerifierCheck_Raw`$(\mathtt{G}, \Delta)$

- **Output:** response-side polynomial value $\mathsf{vc}_{\mathrm{rhs}}$.
- **Definition:** evaluate the polynomial encoded by the response tuple `G` at the point Δ.

Algorithm 6.5. Serialization Procedures

`sig_serialize_signature`(transcript)

- **Input:** the full signer transcript $(e_{\mathrm{eval}}, v_{\mathrm{eval}}, \mathtt{root}, \mathtt{p_poly}, \mathtt{rv_out}, \mathtt{G}, \hat{\mathbf{w}}', \mathtt{hash}, e^{\mathrm{open}}, v^{\mathrm{open}})$.
- **Output:** byte string σ.

`sig_deserialize_signature`(σ)

- **Input:** signature byte string σ.
- **Output:** parsed transcript components, or failure if the signature length or any component encoding is malformed.

6.5 Key Generation

Algorithm 6.6. Algorithm KeyGen

Input: none beyond the fixed instance profile. **Output:** public key `pk`, secret key `sk`.

1. Sample two independent seeds $\mathtt{seed_H}, \mathtt{seed_e} \xleftarrow{\$} \{0,1\}^{8\cdot \mathtt{SEED_BYTES}}$.
2. Rebuild the public state:

$$\mathbf{H} := \mathsf{ExpandMatrix}(\mathtt{seed_H}), \qquad \mathbf{e} := \mathsf{ExpandWitness}(\mathtt{seed_e}), \qquad \mathbf{y} := \mathbf{He}.$$

3. Pack $\mathbf{y}$ to obtain `packed_y` and output

$$\mathtt{pk} := \mathtt{seed_H} \| \mathtt{packed_y}, \qquad \mathtt{sk} := \mathtt{seed_H} \| \mathtt{seed_e}.$$

6.6 Signing

Overview. The signing algorithm realizes the compiler layer of Section 4 composed with the lifted RSD proof system of Section 5. Concretely:

- the commitment phase realizes the vector-commitment layer and the commitment opening proof;

- the interactive-check phase is the concrete realization of the regularity and embedded-consistency checks built over authenticated values;
- the quotient-response phase produces the low-degree polynomial response checked by the verifier;
- the masking and opening phases realize the deferred VOLE opening and the queried Merkle openings required by the compiler.

Algorithm 6.7. Algorithm Sign$(\mathtt{sk}, M)$

Input: secret key $\mathtt{sk}$, message $M \in \{0,1\}^{\leq 2^{63}}$. **Output:** signature σ.

1. **Key recovery and state reconstruction.** Parse $\mathtt{sk} = \mathtt{seed_H} \| \mathtt{seed_e}$, rebuild $(\mathbf{H}, \mathbf{e}, \mathbf{y})$, and invoke `sig_build_public_state` to obtain the embedded state $(\hat{\mathbf{H}}, \hat{\mathbf{e}}', \bar{\mathbf{y}}, \hat{\mathbf{y}}_{\mathbf{e}})$. Reconstruct
$$\mathtt{pk} := \mathtt{seed_H} \| \mathtt{packed_y}.$$
2. **Message hash.** Compute $\mathsf{mh} := \mathsf{Hash}(M)$ and form the transcript prefix $\mathsf{mh} \| \mathtt{pk}$.
3. **Commitment construction.** Execute `Commitment_Raw`. This subprocedure performs the following operations:
   (a) sample a random tail of length `TAIL_LEN` and append it to the witness vector $\hat{\mathbf{e}}'$;
   (b) for the first `SIGMA_REPETITIONS` tail coordinates, force the Y^{B-1} coefficient to be zero so they can serve as the masked auxiliary values used later in the interactive check;
   (c) sample a uniformly random masking vector $\hat{\mathbf{v}}'$ of length N';
   (d) Reed–Solomon encode the two committed vectors and build the Merkle trees over the encoded pairs, yielding the root tuple `root`.
4. **Commitment challenge and evaluations.** Compute
$$\alpha_{\mathrm{com}} := \mathsf{Hash}(\mathsf{mh} \| \mathtt{pk} \| \mathtt{root}),$$
   derive the evaluation point z via `hash_to_rand_commitment`, and execute `Commitment_Evals_Raw` to obtain $(e_{\mathrm{eval}}, v_{\mathrm{eval}})$.
5. **Precompute masked syndrome terms.** Run `Initial_Computation_Prover_Raw` to compute $\hat{\mathbf{y}}_{\mathbf{v}} := \hat{\mathbf{H}}\hat{\mathbf{v}}'$ for later use in the interactive-check phase.
6. **Interactive-check challenge.** Compute
$$\alpha_{\mathrm{ic}} := \mathsf{Hash}(\mathsf{mh} \| \alpha_{\mathrm{com}} \| e_{\mathrm{eval}} \| v_{\mathrm{eval}}),$$
   and derive the challenge vectors $(\mathbf{r}_s, \gamma_s)_{s<t_\Sigma}$ via `hash_to_rand_InterCheck`.
7. **Interactive-check responses.** Execute `Interactive_Check_Raw_Prover`. For each repetition s, this procedure:
   (a) aggregates the embedded syndrome blocks using the random coefficients $\mathbf{r}_s$;

(b) embeds the corresponding binary linear form in the fixed Y^{B-1} coefficient position;

(c) incorporates the auxiliary tail element indexed by $\texttt{TAIL_INTERACTIVE_OFFSET} + s$ via the scalar γ_s;

(d) outputs the pair $(\texttt{p_poly}_s, \texttt{rv_out}_s)$.

8. **Quotient challenge.** Compute

$$\alpha_{\text{poly}} := \mathsf{Hash}(\mathsf{mh} \| \alpha_{\text{ic}} \| \texttt{p_poly} \| \texttt{rv_out}),$$

and derive (τ, ζ) via `hash_to_rand_PolyRes`.

9. **Quotient-response computation.** Execute `PolyDef_Raw` to fix the base polynomial coefficients, and then call `ProverPolynomialResponse_Final_Raw` to compute the six response polynomials $G_0, \ldots, G_5$. This step is the concrete polynomial response corresponding to the low-degree relation checked in the compiler layer.

10. **Masking challenge.** Compute

$$\alpha_{\text{mask}} := \mathsf{Hash}(\mathsf{mh} \| \alpha_{\text{poly}} \| \texttt{G}),$$

and derive the masking scalar Δ via `hash_to_rand_Mask`.

11. **Masked opening vector.** Execute `Masking_Setup_Raw` to form

$$\hat{\mathbf{w}}' := \Delta \cdot \hat{\mathbf{e}}' + \hat{\mathbf{v}}'.$$

This is the concrete implementation of the deferred VOLE opening relation.

12. **Opening challenge.** Compute

$$\alpha_{\text{open}} := \mathsf{Hash}(\mathsf{mh} \| \alpha_{\text{poly}} \| \hat{\mathbf{w}}'),$$

and derive the queried Merkle positions via `hash_to_rand_Open`.

13. **Merkle opening collection.** Execute `Commitment_Open_Prover_Raw`. This subprocedure:

(a) identifies the Reed–Solomon codeword coordinates to be opened;

(b) copies out the corresponding encoded witness and masking symbols;

(c) determines the minimal set of authentication hashes required to verify the selected leaves in each Merkle tree;

(d) serializes these hashes in transcript order.

14. **Serialization.** Serialize $(\texttt{root}, e_{\text{eval}}, v_{\text{eval}}, \texttt{p_poly}, \texttt{rv_out}, \texttt{G}, \hat{\mathbf{w}}', \texttt{hash}, e^{\text{open}})$ using `sig_serialize_signature` and output the resulting byte string as the signature.

6.7 Verification

Overview. Verification replays the same transcript-hash chain and checks, in order, the commitment opening, the interactive-check relations, and the quotient-response identity. The structure mirrors the organization of the signer-side code and the logical decomposition already given in

Sections 4 and 5.

Algorithm 6.8. Algorithm Verify$(\mathtt{pk}, M, \sigma)$

Input: public key $\mathtt{pk}$, message $M \in \{0,1\}^{\leq 2^{63}}$, signature σ. **Output:** accept or reject.

1. **Parse public key and rebuild public state.** Parse $\mathtt{pk} = \mathtt{seed_H} \| \mathtt{packed_y}$, unpack $\mathbf{y}$, and rebuild $(\mathbf{H}, \hat{\mathbf{H}}, \bar{\mathbf{y}})$ from the public seed. If parsing fails, reject.

2. **Parse signature.** Use `sig_deserialize_signature` to recover the transcript components

$$(\mathtt{root}, e_{\text{eval}}, v_{\text{eval}}, \mathtt{p_poly}, \mathtt{rv_out}, \mathtt{G}, \hat{\mathbf{w}}', h_{\text{open}}, \mathtt{hash}, e^{\text{open}}).$$

   If parsing fails, reject.

3. **Recompute transcript hashes and challenges.** Compute $\mathsf{mh} := \mathsf{Hash}(M)$ and replay the same Fiat–Shamir chain α_{com}, α_{ic}, α_{poly}, α_{mask}, and α_{open}. From these values, derive:

   - the evaluation point z;
   - the interactive-check arrays $(\mathbf{r}_s, \gamma_s)$;
   - the quotient challenge pair (τ, ζ);
   - the masking scalar Δ;
   - the queried Merkle positions.

   Here

$$\alpha_{\text{com}} = \mathsf{Hash}(\mathsf{mh} \| \mathtt{pk} \| \mathtt{root}), \qquad \alpha_{\text{ic}} = \mathsf{Hash}(\mathsf{mh} \| \alpha_{\text{com}} \| e_{\text{eval}} \| v_{\text{eval}}).$$

4. **Commitment-opening verification.** Execute `Commitment_Open_Verifier_Raw`. This procedure:

   (a) recomputes the polynomial evaluation of $\hat{\mathbf{w}}'$ at z;

   (b) checks the evaluation identity $w_{\text{eval}} = \Delta \cdot e_{\text{eval}} + v_{\text{eval}}$;

   (c) Reed–Solomon encodes $\hat{\mathbf{w}}'$;

   (d) verifies that the opened encoded symbols match the queried leaves;

   (e) recomputes the Merkle roots from the opened leaves and authentication hashes and checks them against `root`.

   If any of these checks fails, reject.

5. **Interactive-check verification.** First compute

$$\hat{\mathbf{y}}_{\mathbf{w}} := \hat{\mathbf{H}} \hat{\mathbf{w}}'$$

   using `Initial_Computation_Verify_Raw`. Then execute `Interactive_Check_Raw_verify`. For each repetition s, this verifier:

   (a) recomputes the aggregated syndrome linear form from $\mathbf{r}_s$;

   (b) recomputes the embedded coefficient contribution in the Y^{B-1} slot;

(c) checks that the response identity involving $\texttt{p_poly}_s$, $\texttt{rv_out}_s$, Δ, and the masked tail coordinate holds;

(d) checks that the Y^{B-1} coefficient of $\texttt{p_poly}_s$ is zero.

If any repetition fails, reject.

6. **Quotient-response verification.** Execute `PolyDef_Raw`, compute

$$\mathsf{vc}_{\mathrm{lhs}} := \texttt{Polynomial_Challenge_Raw}(\tau, \zeta, \hat{\mathbf{w}}', \Delta)$$

and

$$\mathsf{vc}_{\mathrm{rhs}} := \texttt{VerifierCheck_Raw}(\texttt{G}, \Delta).$$

Accept if and only if $\mathsf{vc}_{\mathrm{lhs}} = \mathsf{vc}_{\mathrm{rhs}}$.

Remark 6.1 (Fiat–Shamir consistency). The verifier replays exactly the same transcript-hash chain as the signer. In particular, the masking scalar Δ is derived only after the commitment data, interactive-check responses, and quotient-response polynomials have all been fixed, and the Merkle opening positions are derived only after $\hat{\mathbf{w}}'$ has been fixed.

Remark 6.2 (No failure probability). Given fixed instance parameters and an honestly generated key pair, $\mathsf{Verify}(\texttt{pk}, M, \mathsf{Sign}(\texttt{sk}, M)) = \mathsf{accept}$ with probability 1 over the signing randomness, provided the deterministic seed expansion, transcript hashing, and serialization interfaces are instantiated as specified.

Remark 6.3 (Key Reuse Bound). The signing algorithm does not update the key material and does not consume any stateful per-key nonce space. Therefore the same key pair may be reused for at least 2^{64} distinct messages under the submitted usage profile, with fresh prover-side randomness sampled independently for each invocation of $\mathsf{Sign}(\texttt{sk}, M)$.

7 Performance Analysis

7.1 Complexity Analysis

Signing. The dominant cost of signing comes from the commitment phase (Reed–Solomon encoding and Merkle tree construction), which requires $O(m_{\mathrm{VC}} \cdot \log N_{\mathrm{VC}})$ hash calls. The regularity and syndrome proof steps each require $O(nB)$ field operations. The overall signing complexity is $O(m_{\mathrm{VC}} \log N_{\mathrm{VC}} + nB)$.

Verification. The verifier must expand $\hat{\mathbf{H}}$ from `seed_H` ($O(n\mu)$ operations), verify Q_{VC} Merkle openings ($O(Q_{\mathrm{VC}} \log N_{\mathrm{VC}})$ hashes), and check the polynomial proximity proof. The regularity check requires $O(nB)$ field operations and the syndrome check requires $O(\mu n)$ field operations. Overall verification complexity: $O(n\mu + Q_{\mathrm{VC}} \log N_{\mathrm{VC}} + nB)$.

Comparison with FAEST-style schemes. The FAEST-VOLEitH paradigm (as in FAEST) incurs prover hash cost $H_{\mathrm{FAEST}}(n, N, \lambda) = \Theta(nST)$ where $T = \lceil \lambda / \log_2 S \rceil$ repetitions and challenge space S. The Sigurd scheme reduces this to $H_{\mathrm{ours}}(n, \rho) = \Theta((n + \kappa)/\rho)$ hash calls, which is significantly smaller in typical settings (e.g., $S \approx 2^{10}$, $T \approx 10$).

7.2 Benchmark Results

To obtain benchmark results, we evaluate our reference implementation on an x86_64 platform with a 13th Gen Intel Core(TM) i7-13700 CPU at 2.10 GHz, 32 GB RAM, running Linux on WSL2 (6.6.87.2-microsoft-standard-WSL2). We compile the reference code using gcc 13.1.0 with the standard optimization flag -O2. The resulting cycle counts are listed in Table 3

We also provide optimized implementations for AVX2-capable platforms. For these benchmarks, we use the same machine as described above, this time compiling the code with gcc 13.1.0 and additional optimization flags including -O3, -march=native, -flto, and -fomit-frame-pointer. The resulting performance numbers should therefore be understood as machine-specific results for the current optimized build configuration. Performance results are listed in Table 4

Table 3: Reference implementation benchmark results.

Instance	Target security (bits)		Time (ms)			Cycles ($\times 10^6$)		
	Classical	Quantum	KeyGen	Sign	Verify	KeyGen	Sign	Verify
Sigurd128	128	80	4.558	7.773	3.314	9.62	16.41	7.00
Sigurd256	256	128	24.151	64.479	22.081	51.01	136.18	46.63
Sigurd512	512	256	121.326	544.049	164.940	256.24	1149.03	348.35

Table 4: Optimized implementation benchmark results.

Instance	Target security (bits)		Time (ms)			Cycles ($\times 10^6$)		
	Classical	Quantum	KeyGen	Sign	Verify	KeyGen	Sign	Verify
Sigurd128	128	80	4.343	1.879	0.792	9.17	3.96	1.67
Sigurd256	256	128	23.798	10.703	7.659	50.26	22.60	16.17
Sigurd512	512	256	113.877	87.158	25.164	240.50	184.07	53.14

8 Security Evaluation

8.1 Provable Security

Completeness.

Theorem 8.1 (Completeness)**.** *For every instance I, every $(\mathtt{pk}, \mathtt{sk}) \leftarrow \mathit{KeyGen}(I)$, every message M, and every $\sigma \leftarrow \mathit{Sign}(I, \mathtt{sk}, M)$:*

$$\Pr[\mathit{Verify}(I, \mathtt{pk}, M, \sigma) = \mathsf{accept}] = 1.$$

Proof. For an honestly generated signature, the signer and verifier derive the same public matrix, embedded syndrome instance, Fiat–Shamir challenges, and verification transcript. The vector-commitment proof is accepted by completeness of VC.LinearProof; the regularity and syndrome checks are accepted by Lemmas 5.2 and 5.3. Thus all checks in Section 6.7 accept for the transcript output by Sign, and the verification result is accept. □

EUF-CMA security.

Theorem 8.2 (EUF-CMA Security of Sigurd)**.** *Under the hardness of the* $\mathrm{RSD}(m,k,B)$ *problem (Definition 2.14), the collision resistance of* H *(Definition 2.1), and the programmable random-oracle model for the hash interfaces* H *and* $\mathcal{H}$ *described in Section 2.3, the scheme* Sigurd *is EUF-CMA secure.*

More concretely, for any PPT adversary $\mathcal{A}$ *making at most* q_s *signing queries and* q_h *random-oracle queries, who succeeds in forging a signature with non-negligible probability* ε*, there exists a PPT RSD solver* $\mathcal{B}$ *that solves the* $\mathrm{RSD}(m,k,B)$ *problem with non-negligible probability*

$$\varepsilon - \varepsilon_{\mathrm{VC}} - \varepsilon_{\mathrm{reg}} - \varepsilon_{\Sigma} - \mathrm{negl}(\lambda) - \frac{t_{\mathrm{FS}} q_s q_h}{2^{\lambda}},$$

where t_{FS} *denotes the number of Fiat–Shamir challenges programmed per signing query, and* $\varepsilon_{\mathrm{VC}} = (1-\delta_{\mathrm{VC}})^{Q_{\mathrm{VC}}}$ *(Lemma 4.2),* $\varepsilon_{\mathrm{reg}} = \frac{B+1}{2^{\iota\tau}}$ *(Lemma 5.2), and* $\varepsilon_{\Sigma} = 2^{-\iota t_{\Sigma}} + 2^{-\iota\tau}$ *(Lemma 5.3), with* $\varepsilon_{\mathrm{RS}} \in (0, 1-\sqrt{\rho})$ *being a fixed Reed–Solomon slack parameter and* $\delta_{\mathrm{VC}} \in [0, 1-\sqrt{\rho}-\varepsilon_{\mathrm{RS}}]$ *being the proximity parameter (Lemma 4.2). The term* $\mathrm{negl}(\lambda)$ *includes the SHVZK distinguishing loss, the public-state simulation loss, and the collision-resistance loss for* H *in the Merkle commitment layer.*

Proof. Let G_0 be the real EUF-CMA experiment for Sigurd. In this game, $\mathcal{A}$ receives the honestly generated public key, can query the signing oracle, and wins by outputting a fresh message-signature pair (m^*, σ^*) accepted by the verifier. If $\mathcal{A}$ succeeds in forging a signature, G_0 outputs 1, and otherwise outputs 0. By definition, the success probability of $\mathcal{A}$ in the EUF-CMA experiment is exactly the probability that G_0 outputs 1:

$$\Pr[G_0 \Rightarrow 1] = \mathsf{Adv}_{\Pi}^{\text{EUF-CMA}}(\mathcal{A}).$$

Let G_1 be the experiment in which $\mathcal{B}$ simulates the public key and signing oracle for $\mathcal{A}$. If $\mathcal{A}$ succeeds in forging a signature, G_1 outputs 1, and otherwise outputs 0. The construction of $\mathcal{B}$ is described below.

$\mathcal{B}$ Construction: $\mathcal{B}$ takes as input an RSD instance $(\mathbf{H}, \mathbf{y})$ and outputs a witness $\mathbf{e}^*$ such that $\mathbf{H}\mathbf{e}^* = \mathbf{y}$ and $\mathbf{e}^*$ is regular. The construction of $\mathcal{B}$ proceeds as follows:

- $\mathcal{B}$ simulates the key generation process of Sigurd by embedding the RSD instance $(\mathbf{H}, \mathbf{y})$ into the public key as specified in the key generation algorithm. Then $\mathcal{B}$ provides the simulated public key $\mathtt{pk}$ to $\mathcal{A}$.
- $\mathcal{B}$ initializes the signing-query set $Q_{\mathsf{Sign}} := \emptyset$ and the random oracle query set $Q_{\mathsf{H}} := \emptyset$, $Q_{\mathcal{H}} := \emptyset$.
- When $\mathcal{A}$ makes a signing query about a message m, $\mathcal{B}$ executes the following steps:
  1. $\mathcal{B}$ randomly samples two sets $\mathcal{R}_{\mathsf{H},m}$ and $\mathcal{R}_{\mathcal{H},m}$ from the output space of H and $\mathcal{H}$, respectively.
  2. $\mathcal{B}$ simulates the signing process of Sigurd for the message m by using the SHVZK simulators of the compiled proof system (Theorem 5.4):
     (a) $\mathcal{B}$ uses the elements in $\mathcal{R}_{\mathsf{H},m}$ as the output of H to construct the Merkle tree in the vector-commitment layer and add the corresponding entries to Q_{H}.

(b) When the signing process requires to call $\mathcal{H}$ to derive the Fiat–Shamir challenges, $\mathcal{B}$ programs the random oracle $\mathcal{H}$: it sets $\mathcal{H}(\mathsf{transcript}) = \mathcal{R}_{\mathcal{H},m}$ for the relevant transcript and adds the corresponding entry to $Q_{\mathcal{H}}$.

Because the compiled proof system for RSD is SHVZK, and the challenges are derived from $\mathcal{R}_{\mathcal{H},m}$, which are uniformly random, so $\mathcal{B}$ can simulate the proof without knowing the witness for the RSD instance. The simulated signature is then returned to $\mathcal{A}$.

- When $\mathcal{A}$ makes a random oracle query x to H or $\mathcal{H}$, $\mathcal{B}$ firstly checks if x is already in Q_{H} or $Q_{\mathcal{H}}$. If it is, $\mathcal{B}$ returns the corresponding output. If it is not, $\mathcal{B}$ randomly samples an output for the query, adds the query and its output to the corresponding set, and returns the output to $\mathcal{A}$.

- When $\mathcal{A}$ outputs a forgery (m^*, σ^*), $\mathcal{B}$ first checks whether $m^* \notin Q_{\mathsf{Sign}}$ and whether σ^* is a valid signature for m^* under the public key $\mathtt{pk}$. If either check fails, $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ executes the following steps:

  1. $\mathcal{B}$ calls the extractor of the vector-commitment linear relation proof (Lemma 4.2) to extract the committed witness $\hat{\mathbf{w}}$ from the valid forgery σ^*.
  2. $\mathcal{B}$ then reconstructs the corresponding binary witness candidate $\mathbf{e}^*$ and checks $\mathbf{He}^* = \mathbf{y}$ and the regularity of $\mathbf{e}^*$. If both checks pass, $\mathcal{B}$ outputs $\mathbf{e}^*$ as the solution to the RSD instance. If either check fails, $\mathcal{B}$ aborts.

G_0 is computationally indistinguishable from G_1. Let $\mathcal{M}_{FS}$ be the set of Fiat–Shamir oracle inputs that $\mathcal{B}$ programmed during the signing query simulations. Define the event Bad_{RO} as the event that before $\mathcal{B}$ programs some input $x \in \mathcal{M}_{FS}$, $\mathcal{A}$ has already queried the same input x. If Bad_{RO} does not happen, then the distribution of G_0 is computationally indistinguishable from G_1 by the SHVZK property of the compiled proof system and the public-state simulation argument.

Since $\mathcal{A}$ cannot get any information about the programmed outputs before querying the random oracle, so $\mathcal{A}$ can only guess the programmed input x with probability at most $1/2^{\lambda}$. Since $\mathcal{A}$ can make at most q_h random oracle queries and at most q_s signing queries, where each signing query can lead to at most t_{FS} programmed inputs, so by the union bound, we have

$$\Pr[\mathsf{Bad}_{RO}] \leq \frac{(t_{FS} q_s q_h)}{2^{\lambda}}.$$

So

$$\Pr[G_1 = 1] \geq \Pr[G_0 = 1] - \Pr[\mathsf{Bad}_{RO}] - \mathrm{negl}(\lambda) = \mathsf{Adv}_{\Pi}^{\text{EUF-CMA}}(\mathcal{A}) - \frac{(t_{FS} q_s q_h)}{2^{\lambda}} - \mathrm{negl}(\lambda).$$

$\mathcal{B}$ Success Probability: Next we analyze the success probability of $\mathcal{B}$ in solving the RSD instance under the assumption that $\mathcal{A}$ succeeds in forging a signature in G_1.

Merkle-binding failure event. Let $\mathsf{Bad}_{\mathsf{H}}$ be the event that, in the forged transcript, some Merkle root or authentication path accepted by the verifier can be interpreted as two distinct openings of the same committed position. Such an event yields two distinct inputs to the Merkle hash function with the same digest at the first node where the two authentication computations diverge. Hence

any adversary that makes $\mathsf{Bad}_{\mathsf{H}}$ occur with non-negligible probability can be turned into a collision finder for H. By the collision resistance of H,

$$\Pr[\mathsf{Bad}_{\mathsf{H}}] \leq \mathrm{negl}(\lambda).$$

Conditioned on $\neg\mathsf{Bad}_{\mathsf{H}}$, the Merkle roots in σ^* bind the adversary to a unique set of opened leaves and therefore the vector-commitment extractor can be applied to the accepted transcript.

VC extraction success event. Conditioned on $G_1 = 1 \wedge \neg\mathsf{Bad}_{\mathsf{H}}$, the signature σ^* encodes an accepting Fiat–Shamir transcript of the compiled RSD proof system. For every compiled deferred opening contained in σ^*, apply Lemma 4.2 to the corresponding $\mathsf{VC.LinearProof}$ instance. Let $\mathsf{Good}_{\mathrm{VC}}$ denote the event that all these VC-layer extractions succeed and return committed sender-side values consistent with the accepted openings. By Lemma 4.2,

$$\Pr[G_1 = 1 \wedge \neg\mathsf{Bad}_{\mathsf{H}} \wedge \mathsf{Good}_{\mathrm{VC}}] \geq \Pr[G_1 = 1] - \Pr[\mathsf{Bad}_{\mathsf{H}}] - \varepsilon_{\mathrm{VC}}.$$

On the event $G_1 = 1 \wedge \neg\mathsf{Bad}_{\mathsf{H}} \wedge \mathsf{Good}_{\mathrm{VC}}$, the compiled transcript can be interpreted as a valid accepting transcript of the original $\mathcal{F}_{\mathsf{sVOLE}}^{p,r,\ell}$-hybrid RSD proof system with all deferred VOLE openings fixed consistently. This is exactly the first layer of the compiler extractor of Lemma 4.4. Let $\hat{\mathbf{e}}^*$ denote the embedded witness object determined by this reconstructed hybrid-model transcript.

Regularity failure event. Let $\mathsf{Bad}_{\mathrm{reg}}$ be the event that $G_1 = 1 \wedge \neg\mathsf{Bad}_{\mathsf{H}} \wedge \mathsf{Good}_{\mathrm{VC}}$ occurs but $\hat{\mathbf{e}}^*$ is not block-regular. By Lemma 5.2, an accepting regularity proof for such a non-regular embedded witness can occur with probability at most $\varepsilon_{\mathrm{reg}}$, hence

$$\Pr[\mathsf{Bad}_{\mathrm{reg}}] \leq \varepsilon_{\mathrm{reg}}.$$

Syndrome failure event. Let Bad_{Σ} be the event that $G_1 = 1 \wedge \neg\mathsf{Bad}_{\mathsf{H}} \wedge \mathsf{Good}_{\mathrm{VC}} \wedge \neg\mathsf{Bad}_{\mathrm{reg}}$ occurs but the same embedded witness does not satisfy the embedded syndrome relation. By Lemma 5.3, an accepting syndrome Σ-proof for such an invalid embedded witness can occur with probability at most ε_{Σ}, hence

$$\Pr[\mathsf{Bad}_{\Sigma}] \leq \varepsilon_{\Sigma}.$$

Overall success probability. Therefore, on the event

$$G_1 = 1 \wedge \neg\mathsf{Bad}_{\mathsf{H}} \wedge \mathsf{Good}_{\mathrm{VC}} \wedge \neg\mathsf{Bad}_{\mathrm{reg}} \wedge \neg\mathsf{Bad}_{\Sigma},$$

the reconstructed hybrid-model witness object is block-regular and satisfies the embedded syndrome equation. By Lemma 5.1, $\mathcal{B}$ can convert $\hat{\mathbf{e}}^*$ into a binary vector $\mathbf{e}^*$ satisfying $\mathbf{H}\mathbf{e}^* = \mathbf{y}$ and the required block-regularity condition. Let $\mathsf{Success}$ denote this event that $\mathcal{B}$ outputs such a valid RSD witness. By a union bound,

$$\begin{aligned}
\Pr[\mathsf{Success}] &\geq \Pr[G_1 = 1 \wedge \neg\mathsf{Bad}_{\mathsf{H}} \wedge \mathsf{Good}_{\mathrm{VC}}] - \Pr[\mathsf{Bad}_{\mathrm{reg}}] - \Pr[\mathsf{Bad}_{\Sigma}] \\
&\geq \Pr[G_0 = 1] - \frac{t_{FS} q_s q_h}{2^{\lambda}} - \Pr[\mathsf{Bad}_{\mathsf{H}}] - \varepsilon_{\mathrm{VC}} - \varepsilon_{\mathrm{reg}} - \varepsilon_{\Sigma} - \mathrm{negl}(\lambda) \\
&\geq \mathsf{Adv}_{\Pi}^{\text{EUF-CMA}}(\mathcal{A}) - \frac{t_{\mathrm{FS}} q_s q_h}{2^{\lambda}} - \varepsilon_{\mathrm{VC}} - \varepsilon_{\mathrm{reg}} - \varepsilon_{\Sigma} - \mathrm{negl}(\lambda).
\end{aligned}$$

This is exactly the claimed lower bound on the success probability of the RSD solver $\mathcal{B}$. □

8.2 Concrete Security of the Underlying Hard Problem

The underlying hard problem is binary Regular Syndrome Decoding (RSD), where the error vector is partitioned into w blocks of size n/B and each block contains exactly one nonzero position. The concrete security of the submitted instances is therefore evaluated by the bit complexity of the best known attacks against the corresponding RSD parameter sets, reported separately in the classical and quantum settings.

To assess this hardness, we consider the main attack families that are currently relevant in the literature, including Regular-ISD variants [9], CCJ-type and generalized-birthday/Wagner-style attacks [7], algebraic attacks [6], and hybrid attacks [16]. Table 5 and Table 6 summarize the security estimates for the three selected parameter sets. In the classical table, each family entry is the smallest classical bit complexity among the corresponding attacks in that family. In the conservative quantum table, Grover-style speedups are applied only to the outer guessing or restart part when such a part can be isolated, while the inner structured work, such as list merging, linear algebra, or Gröbner-basis-type algebraic solving, is not automatically granted an additional quantum speedup; for this reason, the quantum estimates are not obtained by simply halving the classical costs. Under this evaluation methodology, the selected parameters are benchmarked against the strongest attacks among the considered attack families in both the classical setting and a conservative quantum setting.

Table 5: Classical security of the three RSD instances

(m, k, w)	Regular-ISD	CCJ	GBA	Hybrid	AGB2.0	Best classical
$(1302, 738, 217)$	165.76	193.57	128.20	170.96	223.20	128.20
$(2748, 1564, 458)$	338.73	398.58	256.38	351.11	418.25	256.38
$(5676, 3230, 946)$	692.76	797.49	512.23	714.04	807.34	512.23

Table 6: Conservative quantum security of the three RSD instances

(m, k, w)	Regular-ISD	CCJ	GBA	Hybrid	AGB2.0	Best quantum
$(1302, 738, 217)$	104.02	124.32	128.20	170.96	223.20	104.02
$(2748, 1564, 458)$	202.92	224.88	256.38	351.11	418.25	202.92
$(5676, 3230, 946)$	387.24	422.47	512.23	714.04	807.34	387.24

9 Scheme Characteristics and Potential Applications

9.1 Algorithmic Features

We summarize the main algorithmic features of Sigurd as concrete claims.

1. **The VOLEitH layer is generated without GGM trees.** Standard FAEST-style VOLEitH signatures realize VOLE correlations through a GGM-tree-based all-but-one commitment followed by a SoftSpokenOT-style transformation. Sigurd removes both layers. The signer commits directly to the VOLE sender vectors and proves the opened relation $\mathbf{w} = \Delta\mathbf{u} + \mathbf{v}$ with a Reed–Solomon vector commitment and a DEEP-quotient linear-relation proof. In the notation of Section 7, the resulting signing cost is $O(m_{\mathrm{VC}} \log N_{\mathrm{VC}} + nB)$.

2. **RSD is the only structured hardness assumption.** The public key consists of a seed-derived binary parity-check matrix and a syndrome, and the secret key determines a block-regular decoding witness. In the random-oracle model, the security reduction in Section 8 converts a successful forgery into an RSD witness for the public instance, except with negligible probability. Thus Sigurd gives a post-quantum signature construction without relying on lattices, isogenies, elliptic curves, or number-theoretic group assumptions.

3. **The RSD witness is proved blockwise, not bitwise.** The proof system compresses each one-hot block of the binary witness into one element of $\mathbb{F}_{2^{\iota\tau}}$. The regularity check becomes the membership test $\hat{\mathbf{e}}_i \in \{1, Y, \ldots, Y^{B-1}\}$, while the syndrome check is reduced to extracting the Y^{B-1} coefficient of $\hat{\mathbf{H}}\hat{\mathbf{e}}$. This is the mechanism that makes the RSD relation compatible with the GGM-tree-free VOLEitH compiler.

4. **The submitted instances prioritize signing speed over signature size.** On the optimized implementation, signing takes about 1.8 ms, 10.7 ms, and 79.9 ms for NGCC-128, NGCC-256, and NGCC-512. The corresponding signature sizes are about 25 KB, 75 KB, and 283 KB. This is the intended design tradeoff: signer throughput and simple symmetric kernels are favored over compact signatures.

5. **Signing is stateless at the key level.** The key pair is not consumed by a signing operation, and the secret key contains no per-message counter. Reuse of a key pair is supported as long as each signature samples fresh prover-side randomness for the commitment and masking procedures.

9.2 Potential Application Scenarios

1. **High-throughput signing services.** The scheme is particularly suitable for services that must generate a large number of signatures per unit time, such as certificate issuance, log signing, software update authorization, or backend authentication services. In these settings, the signer-side latency and throughput are often more important than minimizing each individual signature to the smallest possible byte length.

2. **Server-side or infrastructure-side signing.** The construction is well matched to deployments in which signatures are generated by servers, gateways, hardware security modules, or other managed infrastructure, while clients mainly verify or store the resulting signatures. Such deployments can usually allocate more computation and bandwidth to the signing service than highly constrained edge devices.

3. **Batch-oriented authentication workflows.** The scheme is relevant for systems that sign many independent records in batches, for example transparency logs, audit trails, telemetry records, or signed database snapshots. The signer can benefit from optimized hashing, Reed–Solomon encoding, and extension-field arithmetic across repeated signing tasks, while the larger signature size can be absorbed by the surrounding storage or transmission layer.

4. **Applications with moderate bandwidth but strict signing-time requirements.** The scheme is best suited to environments where the communication channel or storage layer can tolerate signatures of the submitted sizes, but the signing operation must remain fast and predictable. This includes update distribution, machine-to-machine authorization, and other online services where signing delay is directly visible in system throughput.

References

[1] Andris Ambainis, Mike Hamburg, and Dominique Unruh. Quantum security proofs using semi-classical oracles. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II*, Lecture Notes in Computer Science, pages 269–295. Springer, 2019.

[2] Carsten Baum, Lars Braun, Céline Delpech de Saint Guilhem, Marcel Klooß, Christian Majenz, Swagata Mukherjee, Sebastian Ramacher, Christian Rechberger, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Faest: Algorithm specifications, version 1.0, June 2023.

[3] Carsten Baum, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from vole-in-the-head. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*, Lecture Notes in Computer Science, pages 581–615. Springer, 2023.

[4] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: sampling outside the box improves soundness. *CoRR*, abs/1903.12243, 2019.

[5] Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh V. Vazirani. Strengths and weaknesses of quantum computing. *SIAM J. Comput.*, 26(5):1510–1523, 1997.

[6] Pierre Briaud and Morten Øygarden. A new algebraic approach to the regular syndrome decoding problem and implications for PCG constructions. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, Lecture Notes in Computer Science, pages 391–422. Springer, 2023.

[7] Eliana Carozza, Geoffroy Couteau, and Antoine Joux. Short signatures from regular syndrome decoding in the head. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, Lecture Notes in Computer Science, pages 532–563. Springer, 2023.

[8] Hongrui Cui, Hanlin Liu, Di Yan, Kang Yang, Yu Yu, and Kaiyi Zhang. sfresolved: Shorter signatures from regular syndrome decoding and vole-in-the-head. In Qiang Tang and Vanessa Teague, editors, *Public-Key Cryptography - PKC 2024 - 27th IACR International Conference on Practice and Theory of Public-Key Cryptography, Sydney, NSW, Australia, April 15-17, 2024, Proceedings, Part I*, Lecture Notes in Computer Science, pages 229–258. Springer, 2024.

[9] Andre Esser and Paolo Santini. Not just regular decoding: Asymptotics and improvements of regular syndrome decoding attacks. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VI*, Lecture Notes in Computer Science, pages 183–217. Springer, 2024.

[10] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology - CRYPTO '86,*

Santa Barbara, California, USA, 1986, Proceedings, Lecture Notes in Computer Science, pages 186–194. Springer, 1986.

[11] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions (extended abstract). In *25th Annual Symposium on Foundations of Computer Science, West Palm Beach, Florida, USA, 24-26 October 1984*, pages 464–479. IEEE Computer Society, 1984.

[12] Lov K. Grover. A fast quantum mechanical algorithm for database search. In Gary L. Miller, editor, *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 212–219. ACM, 1996.

[13] Yanpei Guo, Xuanming Liu, Kexi Huang, Wenjie Qu, Tianyang Tao, and Jiaheng Zhang. Deepfold: Efficient multilinear polynomial commitment from reed-solomon code and its application to zero-knowledge proofs. In Lujo Bauer and Giancarlo Pellegrino, editors, *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, pages 3497–3516. USENIX Association, 2025.

[14] Lawrence Roy. Softspokenot: Quieter OT extension from small-field silent VOLE in the minicrypt model. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part I*, Lecture Notes in Computer Science, pages 657–687. Springer, 2022.

[15] Dominique Unruh. Revocable quantum timed-release encryption. *J. ACM*, 62(6):49:1–49:76, 2015.

[16] Tianrui Wang, Anyu Wang, Kang Yang, Hanlin Liu, Yu Yu, Jun Zhang, and Xiaoyun Wang. A hybrid algorithm for the regular syndrome decoding problem. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part IV*, Lecture Notes in Computer Science, pages 466–497. Springer, 2025.

[17] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi, editors, *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, pages 2986–3001. ACM, 2021.

A Security Proofs for VC.LinearProof

A.1 Knowledge Soundness

In this appendix we prove Lemma 4.2. The proof follows the standard DEEP-FRI intuition: an accepting transcript fixes committed codewords that are close to low-degree encodings, and the linear check then forces the opened value to coincide with the unique value consistent with these codewords.

Proof of Lemma 4.2. Fix an adversarial sender $\mathcal{S}^*$. Let Acc denote the event that $\mathcal{S}^*$ makes the verifier accept. Let $\mathsf{Bad}_{\mathrm{RS}}$ denote the event that Acc occurs and at least one of the two quotient words

$$\left(\frac{\tilde{u}_i - U(z)}{L_i - z}\right)_{i\in[N]} \qquad \text{or} \qquad \left(\frac{\tilde{v}_i - V(z)}{L_i - z}\right)_{i\in[N]}$$

has distance at least δN from every degree-$(m-2)$ Reed–Solomon codeword. Let the probability be taken over the verifier's random challenge $\Delta \in \mathbb{F}_{p^r}$ and query set $\mathcal{Q} \subseteq [N]$. Consider commitment objects $c_u = (\mathsf{root}_u, U(z))$ and $c_v = (\mathsf{root}_v, V(z))$ fixed by $\mathcal{S}^*$ before Δ is revealed.

By the straight-line extractability assumption stated in Section 4.1, in the random-oracle model the standard straightline extractor for the Merkle commitments recovers the committed vectors $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{v}}$ corresponding to root_u and root_v. The extractor outputs these two codewords.

We now show that they satisfy the lemma. Because the verifier checks that the quotient word associated with the opened vector is a valid Reed–Solomon codeword, the Reed–Solomon sampling bound implies

$$\Pr[\mathsf{Bad}_{\mathrm{RS}}] \le (1-\delta)^{Q_{\mathrm{VC}}}.$$

Therefore, on the event $\mathsf{Acc} \wedge \neg\mathsf{Bad}_{\mathrm{RS}}$, both quotient words are within distance $< \delta N$ of valid degree-$(m-2)$ Reed–Solomon codewords. By Lemma 2.12, there therefore exist vectors $\mathbf{u}^*$ and $\mathbf{v}^*$ such that

$$\mathrm{dist}(\mathsf{Enc}(\mathbf{u}^*), \tilde{\mathbf{u}}) < \delta N, \qquad \mathrm{dist}(\mathsf{Enc}(\mathbf{v}^*), \tilde{\mathbf{v}}) < \delta N,$$

and the corresponding degree-$(m-1)$ polynomials satisfy $U^*(z) = U(z)$ and $V^*(z) = V(z)$.

Next we prove uniqueness. By Lemma 2.13, for any committed word and a random point $z \in \mathbb{F}_{p^r} \setminus \mathcal{L}$, the probability that there exist two different bounded-degree polynomials whose evaluations are both within distance δN of the committed word and that agree at z is negligible. Hence $\mathbf{u}^*$ and $\mathbf{v}^*$ are unique except with negligible probability.

It remains to prove that any accepted opening must equal $\Delta\mathbf{u}^* + \mathbf{v}^*$. Define

$$\tilde{\mathbf{w}} := \Delta\tilde{\mathbf{u}} + \tilde{\mathbf{v}}.$$

If there exists a vector $\mathbf{w}^\dagger \neq \Delta\mathbf{u}^* + \mathbf{v}^*$ that can still be verified with probability greater than $(1-\delta)^{Q_{\mathrm{VC}}} + \mathrm{negl}(\lambda)$, then the distance between its codeword $\mathsf{Enc}(\mathbf{w}^\dagger)$ and $\tilde{\mathbf{w}}$ does not exceed δN. On the other hand, by definition of $\mathbf{u}^*$ and $\mathbf{v}^*$, the distance between $\mathsf{Enc}(\Delta\mathbf{u}^* + \mathbf{v}^*)$ and $\tilde{\mathbf{w}}$ also does not exceed δN. Consequently, there exist two distinct degree- $(m-1)$ polynomials $T(X)$ and $W^*(X)$ such that both are within distance δN of $\tilde{\mathbf{w}}$ on $\mathcal{L}$ and satisfy

$$T(z) = W^*(z) = \Delta U(z) + V(z).$$

This contradicts Lemma 2.13. Therefore the sender can only send the correct opened vector $\mathbf{w}^* = \Delta\mathbf{u}^* + \mathbf{v}^*$.

Combining the straight-line extraction step with the Reed–Solomon sampling bound and the negligible uniqueness loss, we conclude that the extractor outputs committed codewords satisfying the two stated conditions with probability at least

$$\Pr[\mathsf{Acc}] - (1-\delta)^{Q_{\mathrm{VC}}} - \mathrm{negl}(\lambda).$$

□

A.2 Special Honest-Verifier Zero Knowledge

In this appendix we prove Lemma 4.3. The simulator programs the commitment digest and then answers the verifier's queries with values sampled from the same affine distribution as in a real execution.

Proof of Lemma 4.3. Let the verifier's challenge $\Delta \in \mathbb{F}_{p^r}$ be honestly sampled. We construct a simulator $\mathsf{Sim} = (\mathsf{Sim}_0, \mathsf{Sim}_1)$ for the commitment and proof transcript. The simulator is given the common input opened vector $\mathbf{w} \in \mathbb{F}_{p^r}^{\ell}$ and must sample only the random padding part of the opened word.

Simulator Sim_0. The simulator samples two strings uniformly from the output range of the random oracle and sets them as the Merkle roots root_u and root_v. After receiving the verifier's outside point $z \in \mathbb{F}_{p^r} \setminus \mathcal{L}$, it samples two polynomials $U^*(X), V^*(X)$ of degree at most $m-1$ uniformly at random from $\mathbb{F}_p[X]$ and $\mathbb{F}_{p^r}[X]$, respectively, and sends

$$c_u := (\mathsf{root}_u, U^*(z)), \qquad c_v := (\mathsf{root}_v, V^*(z)).$$

Simulator Sim_1. After receiving Δ, the simulator obtains the public opened prefix $\mathbf{w}$ and samples a length-$(Q_{\mathrm{VC}} + r)$ vector $\mathbf{w}'$ such that, if $W(X)$ is the polynomial whose coefficient vector is $\mathbf{w}^* = \mathbf{w} \| \mathbf{w}'$, then

$$W(z) = \Delta U^*(z) + V^*(z).$$

The simulator sends $\mathbf{w}^*$ to the verifier. After receiving the query set $\mathcal{Q}$, it samples $\tilde{u}_i$ independently and uniformly from $\mathbb{F}_p$ for every $i \in \mathcal{Q}$ and sets

$$\tilde{v}_i = \tilde{w}_i - \Delta \tilde{u}_i \in \mathbb{F}_{p^r},$$

where $\tilde{\mathbf{w}} = \mathsf{Enc}(\mathbf{w}^*)$. It then constructs codewords $\tilde{\mathbf{u}}$ and $\tilde{\mathbf{v}}$ by setting the queried coordinates to these sampled values and sampling all unopened coordinates uniformly at random. Finally, it programs the random oracle so that the Merkle paths open to the previously sampled roots.

We show that the simulated view is computationally indistinguishable from a real execution. First, the first-round messages in Sim_0 have the same distribution as in a real execution, since the sampled roots have the same distribution as random-oracle outputs.

Now fix $\mathbf{u}$ and $\mathbf{v}$, and consider a real honest prover. For every queried position $i \in \mathcal{Q}$,

$$\tilde{u}_i = \sum_{j=0}^{m-1} u_j L_i^j, \qquad U(z) = \sum_{j=0}^{m-1} u_j z^j.$$

Hence both $\tilde{u}_i$ and $U(z)$ are affine functions of the random padding coordinates of $\mathbf{u}'$. The tuple

$$\big(\{\tilde{u}_i\}_{i\in\mathcal{Q}}, U(z)\big)$$

consists of $Q_{\mathrm{VC}} + r$ independent $\mathbb{F}_p$-linear forms in these $Q_{\mathrm{VC}} + r$ random variables, and is therefore uniform over $\mathbb{F}_p^{Q_{\mathrm{VC}}+r}$. By the same argument,

$$\big(\{\tilde{v}_i\}_{i\in\mathcal{Q}}, V(z)\big)$$

is uniform over $(\mathbb{F}_{p^r})^{Q_{\mathrm{VC}}+1}$.

Now condition on fixed values of $U(z)$, $V(z)$, and $\{(\tilde{u}_i, \tilde{v}_i)\}_{i\in\mathcal{Q}}$. In a real execution, the random padding part $\mathbf{w}'$ is constrained only by the affine conditions

$$\tilde{w}_i = \Delta\tilde{u}_i + \tilde{v}_i \quad \text{for all } i \in \mathcal{Q}, \qquad W(z) = \Delta U(z) + V(z).$$

Therefore $\mathbf{w}'$ is uniformly distributed over the corresponding affine subspace, which is exactly how Sim_1 samples it. It follows that the joint distribution of

$$U(z),\ V(z),\ \{(\tilde{u}_i, \tilde{v}_i)\}_{i\in\mathcal{Q}},\ \mathbf{w}^*$$

in the simulation is identical to that in a real execution.

The simulator also enforces the pointwise relation

$$\tilde{w}_i = \Delta\tilde{u}_i + \tilde{v}_i$$

by construction, so the local linear checks pass. Because $W(z) = \Delta U(z) + V(z)$, the quotient

$$Q^*(X) = \frac{W(X) - (\Delta U(z) + V(z))}{X - z}$$

has degree at most $m - 2$, so the Reed–Solomon quotient check also accepts.

Finally, let Bad denote the event that the verifier queries either an unopened leaf value or an internal hash input on which the simulated Merkle tree and the real Merkle tree differ. Since the unopened leaves contain fresh random field elements and the verifier makes only polynomially many random-oracle queries, the probability of Bad is negligible. Conditioned on $\neg\mathsf{Bad}$, the simulated authentication paths are identically distributed to the real ones. Therefore the verifier's overall view in the simulation is computationally indistinguishable from its view in a real execution. □

B Proof of Compiler Security

In this appendix we prove Lemma 4.4. The proof follows the same modular pattern as the compiler construction in this document: correctness is immediate, knowledge extraction first peels off the commitment layer, and simulation is obtained by combining the simulator of the hybrid-model proof with the simulator of the vector-commitment layer.

Proof of Lemma 4.4. **Completeness.** Assume first that the underlying protocol Π is complete. In the compiled protocol, each time the prover would have sent init to the ideal functionality, $\mathcal{P}_{\mathsf{Comp}}$ instead samples $(\mathbf{u}_{sid}, \mathbf{v}_{sid})$ locally, commits to them, and hands exactly the same pair to the internal copy of $\mathcal{P}$. Later, when the verifier issues a get request, $\mathcal{P}_{\mathsf{Comp}}$ computes $\mathbf{w}_{sid} = \Delta_{sid}\mathbf{u}_{sid} + \mathbf{v}_{sid}$ and proves the relation using $\mathsf{VC.LinearProof}$. By completeness of the vector-commitment layer, verification succeeds with probability 1. Hence the internal copies of $\mathcal{P}$ and $\mathcal{V}$ see exactly the same VOLE distribution as in the ideal hybrid model, and the compiled protocol is complete.

Public-coin property. If Π is public-coin, then the only additional verifier messages introduced by the compiler are the random challenges Δ_{sid} sent in response to get. These are sampled uniformly and independently from $\mathbb{F}_{p^r}$. Therefore the compiled verifier remains public-coin.

Knowledge soundness. Assume now that Π has a black-box extractor Ext_Π with knowledge error $\kappa(\lambda)$. Let $\mathcal{P}^*_{\mathsf{Comp}}$ be a malicious compiled prover that convinces the honest compiled verifier with non-negligible probability.

Because Π is sVOLE-admissible, all messages of the form $(sid, \mathsf{get}, \Delta_{sid})$ occur only after the ordinary interaction is finished. Let the last ordinary round be round k. We construct a prover $\mathcal{P}^*$ against the original hybrid-model protocol as follows:

- during the first k rounds, $\mathcal{P}^*$ runs $\mathcal{P}^*_{\mathsf{Comp}}$ internally;
- whenever $\mathcal{P}^*_{\mathsf{Comp}}$ sends a commitment pair $(\mathsf{com}^u_{sid}, \mathsf{com}^v_{sid})$, use the extractor for the vector-commitment layer to recover the committed vectors $(\mathbf{u}_{sid}, \mathbf{v}_{sid})$ and provide them to the ideal functionality as the sender input;
- when the hybrid verifier later queries get and receives $(\Delta_{sid}, \mathbf{w}_{sid})$ from the ideal functionality, forward Δ_{sid} to the internal copy of $\mathcal{P}^*_{\mathsf{Comp}}$ and compare its claimed opening with the ideal value.

By Lemma 4.2, except with probability at most $\varepsilon_{\mathrm{VC}} + \mathrm{negl}(\lambda)$, any accepting linear-relation proof produced by $\mathcal{P}^*_{\mathsf{Comp}}$ is consistent with the extracted $\mathbf{u}_{sid}, \mathbf{v}_{sid}$ and therefore with the ideal value $\mathbf{w}_{sid} = \Delta_{sid}\mathbf{u}_{sid} + \mathbf{v}_{sid}$. Consequently, the success probability of $\mathcal{P}^*$ in the hybrid model differs from the success probability of $\mathcal{P}^*_{\mathsf{Comp}}$ in the compiled model by at most $\varepsilon_{\mathrm{VC}} + \mathrm{negl}(\lambda)$.

Applying Ext_Π to $\mathcal{P}^*$ yields a witness with probability at least $\Pr[\mathcal{P}^*_{\mathsf{Comp}} \text{ accepts}] - \kappa(\lambda) - \varepsilon_{\mathrm{VC}} - \mathrm{negl}(\lambda)$, which establishes knowledge soundness of the compiled protocol with error $\kappa(\lambda) + \varepsilon_{\mathrm{VC}} + \mathrm{negl}(\lambda)$.

SHVZK. Assume finally that Π has a black-box SHVZK simulator Sim_Π. Let $\mathcal{V}^*_{\mathsf{Comp}}$ be a semi-honest verifier for the compiled protocol. We construct a simulator $\mathsf{Sim}_{\mathsf{Comp}}$.

First, extract from $\mathcal{V}^*_{\mathsf{Comp}}$ the randomness determining its ordinary public-coin messages and the later deferred challenges Δ_{sid}. Using this information, define a corresponding semi-honest verifier $\mathcal{V}^*$ for the hybrid-model protocol that sends the same ordinary messages and, upon each deferred get, uses the pair $(\Delta_{sid}, \mathbf{w}_{sid})$ supplied by the simulator.

Run Sim_Π against $\mathcal{V}^*$ to obtain a simulated hybrid transcript. For every commitment pair that must appear in the compiled transcript, invoke the simulator of Lemma 4.3 to generate simulated commitment objects and simulated linear-relation proofs consistent with the same deferred challenge Δ_{sid} and the same simulated $\mathbf{w}_{sid}$.

We now argue indistinguishability by hybrids:

(1) replace the real hybrid-model proof transcript by the output of Sim_Π; this is indistinguishable by SHVZK of Π;

(2) replace each honest linear-relation proof by the simulator from Lemma 4.3; this is indistinguishable by SHVZK of the vector-commitment layer;

(3) observe that the distribution of the deferred VOLE outputs is unchanged, because both in the real and simulated executions the pair $(\mathbf{u}_{sid}, \mathbf{v}_{sid})$ is uniform and $\mathbf{w}_{sid}$ is sampled subject only to $\mathbf{w}_{sid} = \Delta_{sid}\mathbf{u}_{sid} + \mathbf{v}_{sid}$.

Thus the simulated compiled transcript is computationally indistinguishable from a real honest execution, proving SHVZK. □

C Proofs for the RSD Proof System

C.1 Embedding Preserves the Syndrome

Detailed proof of Lemma 5.1. Fix a row block index r and write the embedded row-block product as

$$\sum_{c=0}^{n-1} \hat{\mathbf{H}}^{(r,c)} \hat{\mathbf{e}}_c.$$

Since the witness is regular, each block $\hat{\mathbf{e}}_c$ is a monomial Y^{a_c} for some $a_c \in \{0, \ldots, B-1\}$. Therefore multiplication by $\hat{\mathbf{e}}_c$ shifts the coefficients of $\hat{\mathbf{H}}^{(r,c)}$ upward by a_c positions in the Y-basis. By construction,

$$\hat{\mathbf{H}}^{(r,c)} = \sum_{s=0}^{B-1} \left(\sum_{t=0}^{\iota-1} H^{(r,c)}_{t,B-1-s} X^t \right) Y^s.$$

Hence the coefficient of Y^{B-1} in $\hat{\mathbf{H}}^{(r,c)} Y^{a_c}$ is exactly

$$\sum_{t=0}^{\iota-1} H^{(r,c)}_{t,a_c} X^t,$$

which corresponds to selecting the a_c-th column of the binary block $\mathbf{H}^{(r,c)}$.

Now consider reduction modulo $g(Y)$. Since every term appearing before reduction has Y-degree at most $(B-1)+(B-1) = 2B-2$, and since $g(Y)$ has leading term Y^τ with the next nonzero term of degree at most $\tau - B$, every reduction step lowers degree by at least B. Because $\tau > B$, no term produced by modular reduction can alter the coefficient of Y^{B-1}. Therefore the Y^{B-1} coefficient is identical before and after reduction in $\mathbb{F}_{2^{\iota\tau}}$.

Summing over all column blocks c therefore yields, in the coefficient of Y^{B-1}, exactly the packed row-block product of the original binary matrix with the selected coordinates of the witness. This is the embedded block $\bar{\mathbf{y}}^{(r)} \in \mathbb{F}_{2^\iota}$ of the syndrome. Since r was arbitrary, the claim holds for every row block. □

C.2 Security of the Regularity Protocol

Detailed proof of Lemma 5.2. **Completeness.** Assume the prover holds a valid embedded witness $\hat{\mathbf{e}}$, so that $F(\hat{\mathbf{e}}_i) = 0$ for every $i \in [0, n)$. For each block, the polynomial expansion

$$F(\hat{\mathbf{e}}_i \Delta + v_{\mathbf{e},i}) = c_B^{(i)} \Delta^B + \cdots + c_0^{(i)}$$

has leading coefficient $c_B^{(i)} = F(\hat{\mathbf{e}}_i) = 0$. The prover's response coefficients $(c_0, \ldots, c_{B-1})$ are exactly the coefficients of the verifier's aggregated quantity

$$\sum_{i=0}^{n-1} r_i F(w'_{\mathbf{e},i}) + \sum_{j=0}^{B-2} w_{\text{lookup},j} \Delta^{B-2-j},$$

so the verifier accepts.

SHVZK. The verifier's view consists of the challenge vector $\mathbf{r}$ and the prover's response coefficients $(c_0, \ldots, c_{B-1})$. All witness dependence enters only through masked values of the form $w'_{\mathbf{e},i} = \hat{\mathbf{e}}_i \Delta + v_{\mathbf{e},i}$, where the tags $v_{\mathbf{e},i}$ are uniformly random subject to the IT-MAC relation. Consequently the distribution of the aggregated coefficients is statistically hidden by the random masks already present in the sVOLE functionality. A simulator therefore samples uniformly random authenticated shares consistent with the public challenge $\mathbf{r}$ and computes $(c_0, \ldots, c_{B-1})$ exactly as the honest prover would. This yields a distribution identical to the honest one.

Soundness. Suppose the prover convinces the verifier with a witness for which at least one block $\hat{\mathbf{e}}_{i^*}$ is not in $\{1, Y, \ldots, Y^{B-1}\}$. Then $F(\hat{\mathbf{e}}_{i^*}) \neq 0$, so in the polynomial expansion of $F(\hat{\mathbf{e}}_{i^*}\Delta + v_{\mathbf{e},i^*})$ the coefficient of Δ^B is nonzero. After aggregation with a random challenge vector $\mathbf{r}$, the verifier's acceptance condition becomes a nonzero polynomial identity in the random field elements $r_0, \ldots, r_{n-1}, \Delta$. Its degree in the random coefficients is at most $B+1$, and therefore by Schwartz–Zippel it can hold with probability at most $(B+1)/|\mathbb{F}_{2^{\iota\tau}}| = (B+1)/2^{\iota\tau}$.

Thus the protocol is complete, SHVZK, and has soundness error at most $(B+1)/2^{\iota\tau}$. □

C.3 Security of the Σ-Protocol

Detailed proof of Lemma 5.3. **Completeness.** For an honest prover, the syndrome recovery lemma implies that the Y^{B-1} coefficient of each entry of $\hat{\mathbf{H}}\hat{\mathbf{e}}$ is the corresponding entry of $\bar{\mathbf{y}}$. Therefore the first verifier check,

$$\mathrm{coeff}_{Y^{B-1}}(h'_{\mathbf{e}}) = \bar{y}',$$

holds. The second check follows from the IT-MAC identity $\mathbf{w}'_{\mathbf{e}} = \Delta\hat{\mathbf{e}} + \mathbf{v}_{\mathbf{e}}$ together with linearity of the matrix multiplication by $\hat{\mathbf{H}}$ and the way the correction $u_\Sigma - r_\Sigma$ is incorporated. Hence an honest execution always accepts.

SHVZK. The prover's first message contains only the difference $u_\Sigma - r_\Sigma$, where r_Σ is uniform subject to the single constraint that its Y^{B-1} coefficient vanishes. The prover's final response adds this same masking term to the linear form in $\hat{\mathbf{H}}\hat{\mathbf{e}}$. Therefore, conditioned on the verifier's honest challenge vector $\mathbf{r}$, the response $h'_{\mathbf{e}}$ is uniformly distributed over the affine subspace of field elements with prescribed Y^{B-1} coefficient, while $h'_{\mathbf{v}}$ is then fixed by the authenticated linear relation. A simulator can sample these values directly and thereby reproduce the verifier's view exactly.

Soundness. Suppose the prover is able to make the verifier accept while the syndrome equation is false. There are two cases.

- If the extracted Y^{B-1} coefficient is incorrect, then the prover must guess the value of the random linear form $\bar{y}' = \sum_{i=0}^{\mu-1} r_i \bar{y}_i$ over $\mathbb{F}_{2^\iota}$. Since the challenge vector $\mathbf{r}$ is uniform in $(\mathbb{F}_{2^\iota})^\mu$, this succeeds with probability at most $2^{-\iota}$.
- If the first check holds but the authenticated linear relation is false, then the prover must satisfy a nontrivial affine relation in $\mathbb{F}_{2^{\iota\tau}}$ involving the hidden mask and global VOLE key. Because the ambient field has size $2^{\iota\tau}$, this can happen with probability at most $2^{-\iota\tau}$.

By a union bound, the soundness error is at most $2^{-\iota} + 2^{-\iota\tau}$.

If the protocol is repeated in parallel t_Σ times using independent fresh auxiliary masks and challenges, the first term is amplified to $2^{-\iota t_\Sigma}$ while the second term remains $2^{-\iota\tau}$. This gives the stated bound. □

C.4 Security of the Composed RSD Proof System

Detailed proof of Theorem 5.4. **Completeness and public coin.** The IT-MACs generation phase is complete by correctness of the ideal sVOLE functionality. Lemma 5.2 and Lemma 5.3 imply completeness of the regularity and Σ phases. The verifier's only nontrivial messages are the uniformly sampled challenges $\mathbf{r}^{\mathrm{reg}}$ and $(\mathbf{r}^{(\Sigma,s)})_{s<t_\Sigma}$, so the entire protocol is public-coin.

sVOLE-admissibility. The prover only plays the sender role in $\mathcal{F}_{\mathsf{sVOLE}}^{2^\iota\tau,1,n+B}$. Moreover, the verifier can be split into a first phase that receives the IT-MAC correction values and all ordinary protocol messages, and a second phase that issues the deferred get request and performs only the final verification checks. Hence the protocol is sVOLE-admissible in the sense of Definition 2.7.

Knowledge soundness. From the IT-MACs generation phase, an extractor directly obtains the embedded witness $\hat{\mathbf{e}}$ from the sender messages together with the correction $\mathbf{u_e} - \hat{\mathbf{e}}$. By Lemma 5.2, except with probability $\varepsilon_{\mathrm{reg}}$, this extracted witness satisfies the block-regularity property. By Lemma 5.3, except with probability ε_Σ, it also satisfies the embedded syndrome equation. By Lemma 5.1, this implies that the corresponding binary witness $\mathbf{e}$ satisfies $\mathbf{He} = \mathbf{y}$.

Therefore the extractor outputs a valid witness for $\mathcal{R}_{\mathrm{RSD}}$ except with probability at most $\varepsilon_{\mathrm{reg}} + \varepsilon_\Sigma$, which is exactly the claimed knowledge error.

SHVZK. The IT-MACs generation phase is perfectly simulatable in the ideal sVOLE-hybrid model by sampling random authenticated shares. The regularity protocol is SHVZK by Lemma 5.2, and the Σ-protocol is SHVZK by Lemma 5.3. Composing these simulators in transcript order therefore gives a simulator for the full hybrid-model proof system. Finally, Lemma 4.4 transfers completeness, public coin, knowledge soundness, and SHVZK to the compiled protocol in the random-oracle model. Since the hybrid-model knowledge error is $\varepsilon_{\mathrm{reg}} + \varepsilon_\Sigma$, the compiled protocol inherits knowledge error $\varepsilon_{\mathrm{reg}} + \varepsilon_\Sigma + \varepsilon_{\mathrm{VC}} + \mathrm{negl}(\lambda)$, where $\varepsilon_{\mathrm{VC}} = (1 - \delta_{\mathrm{VC}})^{Q_{\mathrm{VC}}}$ is the vector-commitment extraction error contributed by Lemma 4.2. □